
C#, .NET and MS SQL Server

A complete study guide — full reconstruction of all 134 lectures,
with practice exercises and self-assessment

Prepared for Yassin Hatem Farid

Programme Custom Development Graduate Program, PwC ETIC Egypt

Source course *Learn API Development with C# .NET and MS SQL Server*

Dominic Tripodi — 24.5 hours, 12 sections, 134 lectures

Coverage Sections 1-12 in full, written from zero

Date 14 September 2026

Contents

Module	Topic	Course sections	Lectures
—	How to use this guide	—	—
1	Foundations and the C# Language	\$1 Introduction, \$2 C# Basics	1-26
2	SQL: Basics, Seed and Intermediate	\$3, \$5, \$6	27-29, 54-57
3	C# Intermediate: Classes, Data Access, Config, JSON and Async	\$4 C# Intermediate	30-53
4	API Basics	\$7 API - Basics	58-82
5	Repository Pattern, Authentication and JWT	\$8 API - Intermediate	83-98
6	Stored Procedures	\$9 Stored Procedures	99-110
7	API Advanced	\$10 API - Advanced	111-127
8	Azure Deployment and Project Readiness	\$11 Azure Deploy, \$12	128-134
A	Java to C# Quick Reference	Appendix	
B	Command Reference	Appendix	
C	Glossary	Appendix	
D	Your First Weeks on the Project	Appendix	

Every module ends with **Practice** (replacing the course's coding exercises and assignments) and **Self-check** (replacing its quizzes). Answers are not printed.

C# .NET Study Guide

How to use this guide

This document reconstructs the entire 134-lecture course as written material, section by section, in the course's own order and using the course's own terminology. It is written on the assumption that you have watched none of it.

Structure. Eight modules map onto the course's twelve sections:

Module	Course sections	Lectures	Video time
1	\$1 Introduction, \$2 C# Crash Course - Basics	1-26	4h 00m
2	\$3 SQL Basics, \$5 SQL Seed, \$6 SQL Intermediate	27-29, 54-57	1h 53m
3	\$4 C# Crash Course - Intermediate	30-53	4h 12m
4	\$7 API - Basics	58-82	4h 24m
5	\$8 API - Intermediate	83-98	4h 11m
6	\$9 Stored Procedures	99-110	1h 50m
7	\$10 API - Advanced	111-127	2h 51m
8	\$11 Azure Deploy, \$12 Close	128-134	1h 07m

Conventions used throughout.

- Every subsection is tagged with the lecture numbers it reconstructs, e.g. (*L19, L20*), so you can jump back to the video for any topic you want to see typed out live.
- Boxes marked **JAVA BRIDGE** map a C# concept onto the Java or Spring Boot equivalent you already know. Use these to move fast; ignore them if they slow you down.
- Boxes marked **ON THE JOB** flag where the course's choice differs from what you are likely to meet on a PwC engagement. These are the things that make you look prepared rather than merely trained.
- Boxes marked **TRAP** are the specific mistakes that produce failing builds, silent bugs, or code-review comments.
- Each module ends with **Practice** (replacing the Coding Exercises you skipped) and **Self-check** (replacing the Quizzes). No answers are printed. If you want them marked, send me your work.

The one architectural idea the whole course is building toward. Every section is a piece of this:

```
HTTP request
  |
  v
Controller          <- routing, model binding, HTTP status codes
  |
  v
Repository / Service <- business logic, behind an interface (IUserRepository)
  |
  v
Dapper or EF Core   <- data access
  |
  v
Stored procedures    <- T-SQL in the database
  |
  v
MS SQL Server / Azure SQL
```

with **DTOs** at the HTTP boundary, **AutoMapper** moving data between DTOs and models, and **JWT bearer tokens** securing the whole thing. If you understand that diagram completely — what lives in each box and why — you have the course.

A note on the two data-access paths. This course teaches *both* Dapper and Entity Framework Core, and deliberately solves the same assignment twice, once with each (lectures 81 and 82). That is unusual and it is to your advantage: you do not yet know which your project uses. Dapper with stored procedures is common in finance, government and performance-sensitive systems; EF Core is the Microsoft default and more common in greenfield builds. Learn both. Module 3 and Module 4 cover each in turn.

Module 1 — Foundations and the C# Language

Course sections 1 and 2. Lectures 1–26, plus Coding Exercises 1–7 and Quizzes 1–3.

1.1 What .NET actually is (L1, L3, L10 “Basics Theory Pt 1”, L13 “Basic Theory Pt 2”)

C# is the language. .NET is the platform: a runtime, a standard library, and a toolchain.

You write C# source. The **Roslyn** compiler emits **IL** (Intermediate Language, sometimes MSIL or CIL) into an assembly — a `.dll` or `.exe`. At runtime the **CLR** (Common Language Runtime) JIT-compiles that IL into native machine code for whatever CPU it is running on. The CLR also provides garbage collection, type safety and exception handling.

JAVA BRIDGE

Java	.NET
JVM	CLR
Bytecode (<code>.class</code>)	IL (<code>.dll</code>)
JDK	.NET SDK
JRE	.NET Runtime
Java standard library	BCL (Base Class Library)
Maven / Gradle	NuGet + MSBuild
<code>pom.xml</code>	<code>.csproj</code>
Maven Central	nuget.org
JAR	Assembly (<code>.dll</code>)
Spring Boot	ASP.NET Core

The mental model transfers almost perfectly. Managed memory, JIT, garbage collection, single-inheritance-plus-interfaces — all the same.

The naming history — learn this properly

It comes up in interviews, and it determines what you are looking at when you open a client’s legacy repository.

- **.NET Framework** (1.0 through 4.8, 2002–2019). Windows only. Still running in a large number of enterprise systems. Final version 4.8.1; it receives security patches but no new features. If a client system was built before ~2017, assume this.
- **.NET Core** (1.0 through 3.1, 2016–2019). The cross-platform, open-source rewrite. Runs on Linux and macOS. Much faster.
- **.NET 5** onward (2020–). Core and Framework converged; Microsoft dropped the word “Core” from the platform name to signal there is now one .NET. Versions 5, 6, 7, 8, 9, 10.

Version **6, 8, 10** — the even numbers — are **LTS** (Long Term Support, three years). Odd numbers are **STS** (Standard Term Support, eighteen months).

ON THE JOB

This course targets **.NET 9**. Enterprises overwhelmingly standardise on LTS, so your project will most likely be on **.NET 8** — possibly .NET 10 if it is brand new. The practical difference for everything in this course is close to zero: the same APIs, the same syntax, the same project structure. Check `<TargetFramework>` in the `.csproj` on day one and mention it in standup; it signals you know the release cadence.

The confusing legacy: **ASP.NET** (no Core) is the old Windows-only web framework. **ASP.NET Core** is the modern one. Dominic uses “.NET Core”, “.NET 9” and “ASP.NET Core” interchangeably. They all mean the modern stack here.

1.2 Environment and tooling (L2 “Install List”, L4 “Install”, L5–L6 Mac/Linux SQL Server, L7 MS SQL Extension, L8 Hotkeys and Macros)

The course’s install list:

Tool	Purpose
.NET 9 SDK	Compiler, runtime, <code>dotnet</code> CLI
VS Code + C# Dev Kit extension	Editor. Deliberately <i>not</i> full Visual Studio
MS SQL Server	Developer Edition on Windows; Docker image on Mac/Linux (L5-L6)
MS SQL extension for VS Code (L7)	Run queries without leaving the editor
Azure Data Studio	The course's SQL GUI
Postman	API testing

Dominic's choice of VS Code over Visual Studio is pedagogically deliberate: Visual Studio hides project creation, package management and builds behind buttons, and people who learn that way cannot debug a broken CI pipeline. Learning the CLI means you understand what the buttons do.

ON THE JOB

Two updates to the course's tooling list.

Azure Data Studio was retired by Microsoft in 2026. The current answers are **SSMS** (SQL Server Management Studio, Windows only, the enterprise standard) or the **MSSQL extension for VS Code** (cross-platform, which the course already installs anyway). If your team uses SSMS, everything the course does in Azure Data Studio works identically there.

Your team may well use Visual Studio, not VS Code. Visual Studio has a far better debugger, EF Core tooling and profiler. Learn the CLI first for understanding, then use whatever the team uses. Do not be the graduate who insists on a different editor.

The `dotnet` CLI — the commands you will use daily

```
dotnet --version           # SDK version in use
dotnet --list-sdks         # all installed SDKs
dotnet --info              # full diagnostic dump

dotnet new console -n MyApp # scaffold a console app
dotnet new webapi -n MyApi  # scaffold a web API
dotnet new classlib -n MyLib # scaffold a class library
dotnet new sln -n MySolution # create a solution file
dotnet new gitignore        # .NET-aware .gitignore
dotnet sln add MyApi/MyApi.csproj

dotnet restore             # download NuGet packages
dotnet build               # compile
dotnet run                 # build and execute
dotnet watch run           # hot reload on file save
dotnet clean               # delete bin/ and obj/

dotnet add package Dapper  # add a dependency
dotnet add package Dapper --version 2.1.35 # pin a version
dotnet list package        # what is referenced
dotnet list package --outdated

dotnet test                # run tests
dotnet publish -c Release -o ./publish # produce deployables
```

JAVA BRIDGE `dotnet restore` $\approx$ `mvn dependency:resolve`. `dotnet build` $\approx$ `mvn compile`. `dotnet run` $\approx$ `mvn spring-boot:run`. `dotnet publish` $\approx$ `mvn package`. `dotnet watch run` $\approx$ Spring DevTools.

Anatomy of a project

Item	What it is
<code>Program.cs</code>	Entry point
<code>MyApp.csproj</code>	The project file. XML. Target framework, package references. Your <code>pom.xml</code>
<code>bin/</code>	Compiled output. Gitignored
<code>obj/</code>	Intermediate build artifacts. Gitignored
<code>MySolution.sln</code>	Groups multiple projects together
<code>appsettings.json</code>	Configuration (Module 3)
<code>NuGet.config</code>	Package feed configuration, if present

A minimal `.csproj`:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Dapper" Version="2.1.35" />
  </ItemGroup>
</Project>
```

`ImplicitUsings` automatically imports the common namespaces (`System`, `System.Collections.Generic`, `System.Linq`, and so on) so you do not write them at the top of every file. `Nullable` turns on the null-safety analysis covered in 1.5.

TRAP When a build goes inexplicably wrong — stale errors, phantom missing types, a package that “should” be there — delete `bin/` and `obj/` and rebuild. `dotnet clean` does it properly. This resolves a genuinely large share of mysterious .NET build failures.

NuGet package source (L16, L17)

`dotnet add package X` pulls from nuget.org by default. A `NuGet.config` file in the repository root can redirect that to a private feed.

ON THE JOB A PwC repository will almost certainly have a `NuGet.config` pointing at an internal **Azure Artifacts** feed, for licensing and supply-chain-security reasons. When `dotnet restore` fails on your first day with a 401 or a “package not found”, that file is the first place to look — you will usually need to authenticate to the feed with the Azure Artifacts Credential Provider. Ask your team lead for the setup steps rather than burning half a day.

1.3 Console applications and output (L11 Console App Theory, L12 Creating a Console App, L14 Console.WriteLine, L15 Console App Template)

`dotnet new console -n HelloWorld` generates, in .NET 9, a `Program.cs` containing one line:

```
Console.WriteLine("Hello, World!");
```

No class. No `Main` method. This is **top-level statements**, introduced in C# 9. The compiler still generates the traditional form behind the scenes:

```
using System;

namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Both are valid. Older tutorials show the long form; new templates produce the short one. Recognise both.

Console I/O

```
Console.WriteLine("with newline");
Console.Write("without newline");
string? input = Console.ReadLine(); // always string?, may be null
Console.Clear();
```

`Console.ReadLine()` returns `string?` — nullable — because stdin can be closed. This is your first encounter with nullable reference types.

String interpolation — the most-used syntax feature in C

```
string name = "Yassin";
int age = 23;

Console.WriteLine($"Name is {name} and age is {age}");
Console.WriteLine($"Next year: {age + 1}"); // expressions inline
Console.WriteLine($"Price: {price:C}"); // C = currency
Console.WriteLine($"Percent: {rate:P2}"); // P2 = 2dp percentage
Console.WriteLine($"Date: {date:yyyy-MM-dd}");
Console.WriteLine($"Padded: {value,10}"); // right-align in 10 chars
Console.WriteLine($"Padded: {value,-10}"); // left-align
```

The `$` prefix is mandatory. Without it the braces are literal text.

Other string forms:

```
// Verbatim string – backslashes and newlines are literal
string path = @"C:\Users\Yassin\Documents";

// Interpolated verbatim – combine them
string full = $"{@"C:\Users\{name}\Documents"}";

// Raw string literal (C# 11+) – no escaping at all
string sql = """
    SELECT UserId, FirstName, LastName
    FROM TutorialAppSchema.Users
    WHERE UserId = @UserId
    """;
```

ON THE JOB Raw string literals are outstanding for the embedded SQL you will write throughout this course. The closing `"""` sets the indentation baseline — everything indented to that level is stripped. Much cleaner than the concatenated-string SQL you will see in older code.

JAVA BRIDGE `$"..."` is Java's `String.format` / text blocks, but inline and compile-time checked. `Console.WriteLine` is `System.out.println`. `@"..."` has no Java equivalent.

1.4 Variables and types (L18 Variables Notes, L19 Variables — 22 min)

Value types vs reference types

This is the concept Java developers most often get wrong, because C# has genuine user-definable value types and Java does not.

Value types hold their data directly. Assignment copies the value. They live on the stack (or inline inside a containing object). `int`, `long`, `short`, `byte`, `float`, `double`, `decimal`, `bool`, `char`, `DateTime`, `Guid`, all `struct`s, all `enum`s.

Reference types hold a reference to data on the heap. Assignment copies the reference. `string`, arrays, all `class`es, `List<T>`, `Dictionary<K,V>`, `object`, delegates.

```
int a = 5;
int b = a;           // independent copy
b = 10;              // a is still 5

var list1 = new List<int> { 1, 2 };
var list2 = list1;    // same object on the heap
list2.Add(3);         // list1.Count is now 3
```

`string` is a reference type but is **immutable** and has value-like equality semantics. Every operation that “modifies” a string returns a new one.

The numeric types

Type	Size	Range / precision	Use for
<code>byte</code>	8-bit	0 to 255	Binary data
<code>short</code>	16-bit	±32,767	Rarely
<code>int</code>	32-bit	±2.1 billion	Default whole number
<code>long</code>	64-bit	±9.2 quintillion	IDs, timestamps, large counts
<code>float</code>	32-bit	~7 digits	Rarely
<code>double</code>	64-bit	~15-17 digits	Scientific, approximate
<code>decimal</code>	128-bit	28-29 digits, base-10	Money. Always.

TRAP — the money bug

`double` is base-2 floating point. It cannot represent 0.1 exactly. `0.1 + 0.2` evaluates to `0.30000000000000004`. Over thousands of transactions this drifts, and in a financial-services engagement that drift reaches an audit.

`decimal` is base-10 and exact for decimal fractions. **Use `decimal` for currency, tax rates, and anything a client will reconcile against a ledger.** Map it to SQL `DECIMAL(18,2)`.

Given the kind of work PwC does, this is not an academic point.

Literal suffixes are required where the default type differs:

```
decimal price = 19.99m;    // m or M – without it, 19.99 is a double and won't compile
long big           = 5000000000L;
float f            = 1.5f;
double d           = 1.5;    // no suffix needed, double is the default
uint u            = 5u;
```

Other core types

```
bool isActive = true;           // only true/false – no truthy integers
char grade = 'A';              // single quotes; double quotes = string
string name = "Yassin";        // double quotes

DateTime now = DateTime.Now;    // local machine time
DateTime utc = DateTime.UtcNow; // UTC
DateTime specific = new DateTime(2026, 9, 14);
DateOnly d = new DateOnly(2026, 9, 14); // .NET 6+, date without time
TimeOnly t = new TimeOnly(14, 30);

Guid id = Guid.NewGuid();      // globally unique identifier
```

ON THE JOB Store and compare timestamps in **UTC**. `DateTime.Now` bakes in the server's timezone, and the moment your app runs in an Azure region in a different zone from the client, your data is wrong. Use `DateTime.UtcNow` for anything persisted; convert to local only for display. Egypt is UTC+2 (UTC+3 in summer), and the mismatch will bite you.

var — implicit typing

```
var name = "Yassin";           // compiler infers string
var count = 5;                 // int
var users = new List<User>();   // List<User>
```

Still statically typed, fully checked at compile time. Identical in spirit to Java's `var`. `var` cannot be used for fields, parameters, or when there is no initialiser.

The convention most teams land on: use `var` when the type is obvious from the right-hand side, explicit types when it is not. Match your team's style guide rather than arguing about it.

const and readonly

```
const int MaxRetries = 3;           // compile-time constant, implicitly static
static readonly DateTime Started = DateTime.Now; // runtime value, assigned once
private readonly IUserRepository _repo; // assigned in constructor, never after
```

`const` must be a compile-time literal. `readonly` can be any value but only assignable in the declaration or a constructor. The `private readonly` field assigned in a constructor is the single most common pattern in .NET dependency injection — you will write it in every class from Module 4 onward.

Nullable reference types

Modern .NET projects enable `<Nullable>enable</Nullable>` by default, and it changes what type declarations mean:

```
string name = null;           // ⚠ compiler warning – string is non-nullable
string? name = null;          // ✓ explicitly nullable
```

The `?` is **compile-time analysis only**. It does not change the runtime type; it instructs the compiler to warn you when you might dereference null. It exists to eliminate `NullReferenceException` — .NET's `NullPointerException` and, by Microsoft's own accounting, the most common crash in .NET software.

```
string? input = Console.ReadLine();
Console.WriteLine(input.Length); // ⚠ warning: may be null
Console.WriteLine(input!.Length); // != "trust me" – suppresses the warning
Console.WriteLine(input?.Length); // safe: returns int? (null if input is null)
Console.WriteLine(input?.Length ?? 0); // safe with a default
```

For **value** types, `?` means something different and stronger — it produces `Nullable<T>`, an actual wrapper:

```
int? age = null;
if (age.HasValue) { Console.WriteLine(age.Value); }
int definite = age ?? 0;
```

ON THE JOB This matters enormously for database work. A **nullable SQL column maps to a nullable C# type**: `NVARCHAR NULL` → `string?`, `INT NULL` → `int?`. Getting this wrong is the single most common source of runtime errors when mapping query results to models. Module 3 returns to it.

The `!` null-forgiving operator is a code smell in most places. Every `!` is you telling the compiler you know better. Sometimes you do — but expect to justify each one in review.

1.5 Operators and conditionals (L21 Operators and Conditionals — 24 min, L22 Basic Theory Pt 3, L23 Conditional Statements)

Arithmetic, comparison and logical operators are identical to Java: `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `&&`, `||`, `!`, `++`, `--`, `+=`, `-=`, `*=`, `/=`.

The differences worth knowing:

```
// Null-coalescing
string display = name ?? "Unknown";    // use "Unknown" if name is null
name ??= "Default";                    // assign only if currently null

// Null-conditional ("Elvis")
int? length = user?.Name?.Length;      // null if user OR Name is null
user?.Save();                           // call only if user is non-null
var first = list?[0];                   // index only if list is non-null

// Ternary – same as Java
string status = isActive ? "Active" : "Inactive";

// String equality – C# == compares CONTENT for strings
string a = "hello";
string b = "hello";
bool same = (a == b);                  // true
```

TRAP — reverse Java instinct In Java you are trained never to use `==` on strings. In C#, `==` is operator-overloaded for `string` to compare content, so `==` is correct and idiomatic. `ReferenceEquals(a, b)` is how you compare references if you ever genuinely need to.

For case-insensitive comparison use `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)` — not `a.ToLower() == b.ToLower()`, which allocates two strings and has culture bugs (the classic being the Turkish dotless `ı`).

Conditionals

```
if (condition) { }
else if (other) { }
else { }
```

The condition **must** be `bool`. C# has no truthy integers — `if (count)` does not compile. This is a deliberate improvement over C and JavaScript.

Switch statements and switch expressions

```
// Classic switch statement – break is mandatory, no fall-through allowed
switch (role)
{
    case "Admin":
        access = 3;
        break;
    case "Manager":
    case "Lead":           // stacked labels are allowed
        access = 2;
        break;
    default:
        access = 0;
        break;
}

// Switch EXPRESSION (C# 8+) – returns a value, must be exhaustive
int access = role switch
{
    "Admin" => 3,
    "Manager" => 2,
    "User" => 1,
    _ => 0           // _ is the discard pattern: the default arm
};
```

Switch expressions are everywhere in modern C#. Learn to read and write them.

Pattern matching

```
// Type pattern – check and bind in one step
if (obj is string s && s.Length > 0)
{
    Console.WriteLine(s.ToUpper());
}

// Negated
if (obj is not null) { }

// Relational and logical patterns in a switch expression
string category = age switch
{
    < 0          => "Invalid",
    < 13         => "Child",
    >= 13 and < 20 => "Teen",
    >= 20 and < 65 => "Adult",
    -           => "Senior"
};

// Property pattern
string shipping = order switch
{
    { Total: > 100, IsPrime: true } => "Free express",
    { Total: > 100 }                 => "Free standard",
    -                               => "Paid"
};
```

1.6 Data structures (L20 Data Structures – 34 min, the longest lecture in this section)

Need	C#	Java
Fixed-size sequence	<code>int[]</code>	<code>int[]</code>
Growable list	<code>List<T></code>	<code>ArrayList<T></code>
Key → value	<code>Dictionary<K,V></code>	<code>HashMap<K,V></code>
Unique set	<code>HashSet<T></code>	<code>HashSet<T></code>
FIFO	<code>Queue<T></code>	<code>LinkedList</code> / <code>ArrayDeque</code>
LIFO	<code>Stack<T></code>	<code>Stack</code> / <code>ArrayDeque</code>
Read-only view	<code>IReadOnlyList<T></code>	<code>List.of()</code>

Arrays

```
int[] numbers = new int[5]; // zero-initialised
int[] numbers = { 1, 2, 3, 4, 5 };
int[] numbers = new int[] { 1, 2, 3 };
int[] numbers = [1, 2, 3]; // C# 12 collection expression

Console.WriteLine(numbers.Length); // property – not .length, not .size()
Array.Sort(numbers);
Array.Reverse(numbers);

int[,] grid = new int[3, 3]; // true 2D array
grid[0, 0] = 1;
int[][] jagged = new int[3][]; // array of arrays
```

Arrays are fixed-size. You want `List<T>` almost always.

List<T>

```
List<string> names = new List<string>();
List<string> names = new List<string> { "Yassin", "Ahmed", "Omar" };
List<string> names = new() { "Yassin" }; // target-typed new – very common

names.Add("Sara");
names.AddRange(otherList);
names.Insert(0, "First");
names.Remove("Ahmed"); // by value
names.RemoveAt(0); // by index
names.RemoveAll(n => n.StartsWith("A"));

Console.WriteLine(names.Count); // .Count – not .size()
Console.WriteLine(names[0]); // indexer – not .get(0)
bool has = names.Contains("Omar");
int idx = names.IndexOf("Omar");
names.Sort();
names.Clear();
```

TRAP `.Count` for collections, `.Length` for arrays and strings. Mixing them up is the most common first-week compile error for Java developers.

Dictionary<TKey, TValue>

```
Dictionary<int, string> users = new Dictionary<int, string>();

users.Add(1, "Yassin");           // throws if key already exists
users[2] = "Ahmed";               // add or update – never throws

// The safe lookup idiom
if (users.TryGetValue(1, out string? found))
{
    Console.WriteLine(found);
}

bool exists = users.ContainsKey(1);
users.Remove(1);

foreach (KeyValuePair<int, string> kvp in users)
{
    Console.WriteLine($"{kvp.Key}: {kvp.Value}");
}
foreach (var (key, value) in users)    // deconstruction – cleaner
{
    Console.WriteLine($"{key}: {value}");
}
```

`TryGetValue` with an `out` parameter is idiomatic C# with no clean Java equivalent. It avoids both a double lookup and an exception. Learn it — it appears constantly.

HashSet<T>, Queue<T>, Stack<T>

```
HashSet<string> unique = new HashSet<string>();
unique.Add("a");           // returns false if already present
unique.UnionWith(other);
unique.IntersectWith(other);

Queue<string> q = new Queue<string>();
q.Enqueue("first");
string next = q.Dequeue();
string peek = q.Peek();

Stack<string> s = new Stack<string>();
s.Push("first");
string top = s.Pop();
```

LINQ — the single highest-value feature in C

The course introduces LINQ gradually rather than in one lecture, but it deserves to be learned early. It is Java Streams, but better integrated and with far less ceremony.

```
// Filtering and projection
var adults = users.Where(u => u.Age >= 18).ToList();
var names = users.Select(u => u.Name).ToList();
var nameAges = users.Select(u => new { u.Name, u.Age }).ToList(); // anonymous type

// Ordering
var sorted = users.OrderBy(u => u.LastName)
    .ThenBy(u => u.FirstName)
    .ToList();
var newest = users.OrderByDescending(u => u.CreatedAt).ToList();

// Single elements
var first = users.FirstOrDefault(u => u.Id == 5); // null if none
var firstEx = users.First(u => u.Id == 5); // THROWS if none
var single = users.SingleOrDefault(u => u.Email == e); // throws if MORE than one

// Aggregation
bool any = users.Any(u => u.IsActive);
bool all = users.All(u => u.IsActive);
int count = users.Count(u => u.IsActive);
decimal sum = orders.Sum(o => o.Amount);
decimal avg = orders.Average(o => o.Amount);
decimal max = orders.Max(o => o.Amount);

// Grouping and set operations
var byDept = users.GroupBy(u => u.Department);
foreach (var group in byDept)
{
    Console.WriteLine($"{group.Key}: {group.Count()}");
}
var distinct = names.Distinct().ToList();
var page = users.Skip(20).Take(10).ToList(); // pagination
```

TRAP — the `OrDefault` decision `First()` throws `InvalidOperationException` if nothing matches. `FirstOrDefault()` returns `null` (or the default for value types). `Single()` additionally throws if *more than one* matches.

Choosing correctly is a genuine code-review point. Use `SingleOrDefault` when a duplicate would indicate a data-integrity bug you want surfaced loudly. Use `FirstOrDefault` when you genuinely just want the first. Never use `First()` on user-supplied input.

TRAP — deferred execution `Where` and `Select` are **lazy**. They build a query and execute nothing until you call `.ToList()`, `.ToArray()`, `.Count()` or iterate. Two consequences: (1) enumerating twice runs the query twice; (2) in EF Core, an un-materialised query is still a database call waiting to happen. When in doubt, materialise with `.ToList()`.

1.7 Loops (L24 Loops — 28 min)

```
for (int i = 0; i < 10; i++) { }

foreach (var user in users) { }    // "in", not Java's colon

while (condition) { }

do { } while (condition);

break;    // exit the loop
continue; // next iteration
```

TRAP You cannot modify a collection while `foreach`-ing over it — it throws `InvalidOperationException`, the analogue of Java's `ConcurrentModificationException`. Either iterate a copy (`foreach (var x in list.ToList())`), collect the items to remove and remove them afterwards, or use `RemoveAll(predicate)`.

1.8 Methods (L25 Methods, L26 Scope)

```
public int Add(int a, int b)
{
    return a + b;
}

// Expression-bodied member – for one-liners
public int Add(int a, int b) => a + b;

// void
public void Log(string message) { }

// Optional parameters
public void Log(string message, string level = "INFO") { }

// Named arguments – call-site clarity
Log(level: "ERROR", message: "Failed");

// Overloading – same name, different signature
public int Add(int a, int b) => a + b;
public double Add(double a, double b) => a + b;

// out – return additional values
public bool TryParse(string input, out int result)
{
    return int.TryParse(input, out result);
}
if (TryParse("42", out int value)) { Console.WriteLine(value); }

// ref – pass by reference
public void Double(ref int x) { x *= 2; }

// params – variable argument count
public int Sum(params int[] numbers) => numbers.Sum();
Sum(1, 2, 3, 4);
```

Access modifiers

Modifier	Visible from
<code>public</code>	Anywhere
<code>private</code>	Containing type only (default for members)
<code>protected</code>	Containing type and derived types
<code>internal</code>	The same assembly (default for top-level types)
<code>protected internal</code>	Same assembly, or derived types anywhere
<code>private protected</code>	Derived types within the same assembly

`internal` is roughly Java's package-private but scoped to the assembly (`.dll`), not the namespace. There is no namespace-level visibility in C#.

Scope

Variables live inside the nearest enclosing braces. A variable declared inside an `if` block does not exist after it. Unlike Java, C# forbids shadowing a local variable in a nested scope — the compiler rejects it, which prevents a whole class of bug.

Naming conventions — get these right

Element	Convention	Example
Class, interface, enum, struct	PascalCase	UserRepository
Method	PascalCase	GetUserById
Property	PascalCase	FirstName
Public field	PascalCase	MaxSize
Parameter, local	camelCase	userId
Private field	<u>camelCase</u>	_dapper, _config
Interface	I + PascalCase	IUserRepository
Constant	PascalCase	MaxRetries
Async method	PascalCase + Async	GetUserAsync

TRAP — the one that marks you Methods are PascalCase in C#. Every Java developer's first C# pull request gets review comments about `getUserById` instead of `GetUserById`. The leading underscore on private fields is equally non-negotiable. Configure your editor's C# analyzers and let them catch it before a reviewer does.

Practice — Module 1

Replaces Coding Exercises 1–7. Create a scratch project: `dotnet new console -n Module1Practice`. Write these yourself; do not look up solutions.

- Console and interpolation.** Read a first name and a birth year from `Console.ReadLine()`. Compute the age. Print a greeting including today's date formatted as `dd MMM yyyy`. Handle a non-numeric year using `int.TryParse` rather than letting it throw.
- Value vs reference.** Write two methods: one takes an `int` and reassigns it, one takes a `List<int>` and adds to it. Call both, print the caller's values afterwards, and write a comment explaining why one changed and the other did not.
- The money bug.** Print `0.1 + 0.2` as a `double` and as a `decimal`. In a comment, state which you would use for an invoice total and why, in one sentence.
- Type suffixes.** Declare a `decimal` price, a `long` ID, and a `float` ratio, all with literal values. Deliberately omit the `m` suffix on the decimal first and record the compiler error you get.
- Collections and LINQ.** Build a `List<string>` of ten names. Using LINQ, produce: (a) those starting with "A"; (b) the list sorted alphabetically; (c) the count longer than five characters; (d) the first name containing "z", without throwing when none exists.
- Dictionary.** Build a `Dictionary<int, string>` of user IDs to names. Write `string Lookup(int id)` returning "Not found" for a missing ID — using `TryGetValue`, with no exception and no double lookup.
- Switch expression.** Write `string GetAccessLevel(string role)` using a switch expression with arms for `Admin`, `Manager`, `User`, and a default.
- Pattern matching.** Write `string Categorise(int age)` using relational patterns in a switch expression: under 13 Child, 13–19 Teen, 20–64 Adult, 65+ Senior.
- Null safety.** Given `string? input`, write a single expression that produces the length of its uppercase form, defaulting to 0 when null, using `?.` and `??`.
- Methods.** Write `bool TryGetDiscount(decimal total, out decimal discount)` returning `true` and setting a 10% discount when the total exceeds 100, otherwise `false` with a zero discount.
- Loops.** Given a `List<int>`, remove every even number — first by trying to do it inside a `foreach` (observe the exception), then correctly.
- Naming audit.** Take everything you wrote above and correct every identifier to C# conventions: PascalCase methods, `_camelCase` private fields, camelCase locals.

Self-check — Module 1

Replaces Quizzes 1–3. Answer from memory, without searching.

- What is the relationship between C#, IL, and the CLR?
- Name the three eras of .NET naming and say which one a 2016-era Windows-only enterprise app belongs to.
- Which .NET versions are LTS, and why does that determine what your project runs?
- What does `dotnet restore` do, and which file tells it where to look for packages?
- Why would `dotnet restore` fail on your first day at a client site?

6. Give two reasons `decimal` is used for money instead of `double`.
7. What does the `m` suffix do, and what happens without it?
8. What is the difference between a value type and a reference type at assignment?
9. What does `string?` change at compile time, and what does it change at runtime?
10. How does `int?` differ from `string?` in what it actually produces?
11. Why is `a == b` correct for comparing strings in C# but wrong in Java?
12. When would you use `SingleOrDefault` rather than `FirstOrDefault`?
13. What does deferred execution mean in LINQ, and what bug does it cause?
14. `.Count` or `.Length` — which for a `List<T>`, and which for a `string`?
15. What does `TryGetValue` give you that `dictionary[key]` does not?
16. Why is `_dapper` written with a leading underscore?
17. Write, from memory, the correct C# name for a method that fetches a user by ID.
18. What is `internal` visibility scoped to?
19. What happens if you modify a list inside a `foreach` over that list?
20. Why should you use `DateTime.UtcNow` rather than `DateTime.Now` for stored timestamps?

Module 2 — SQL: Basics, Seed and Intermediate

Course sections 3, 5 and 6. Lectures 27-29 (SQL Basics + Quiz 4), 54-56 (SQL Seed), 57 (SQL Intermediate, a single 61-minute lecture) + Quiz 7.

The course's dialect is **T-SQL** (Transact-SQL), Microsoft's extension to standard SQL, used by MS SQL Server and Azure SQL. If your SQL background is PostgreSQL or MySQL, most of it transfers, but the differences listed in 2.7 will catch you.

2.1 The database the course builds (L35 Computer SQL Script, L56 SQL Seed)

Everything from here to the end of the course operates on one schema. Know it cold, because every stored procedure, model, DTO and endpoint refers back to it.

Schema name: `TutorialAppSchema`. A schema is a namespace inside a database — a container for tables, views and procedures. Using a named schema rather than the default `dbo` is good practice and the course does it deliberately.

Table	Purpose	Key columns
<code>Users</code>	Core person record	<code>UserId</code> (PK, identity), <code>FirstName</code> , <code>LastName</code> , <code>Email</code> , <code>Gender</code> , <code>Active</code>
<code>UserSalary</code>	One-to-one with Users	<code>UserId</code> (PK/FK), <code>Salary</code> DECIMAL(18,4)
<code>UserJobInfo</code>	One-to-one with Users	<code>UserId</code> (PK/FK), <code>JobTitle</code> , <code>Department</code>
<code>Auth</code>	Credentials, kept separate	<code>Email</code> (PK), <code>PasswordHash</code> VARBINARY, <code>PasswordSalt</code> VARBINARY
<code>Posts</code>	One-to-many from Users	<code>PostId</code> (PK, identity), <code>UserId</code> (FK), <code>PostTitle</code> , <code>PostContent</code> , <code>PostCreated</code> , <code>PostUpdated</code>

Two design decisions worth understanding, because they are the kind of thing a reviewer asks you to justify:

Why is `Auth` a separate table keyed on `Email` rather than columns on `Users`? Separation of concerns and blast radius. A query that returns user profile data can never accidentally leak a password hash, because the hash is not in that table. You can also apply different permissions to the two tables. This is a genuinely good pattern and worth repeating.

Why are `UserSalary` and `UserJobInfo` separate one-to-one tables? Partly to give the course something to demonstrate `JOIN` and `OUTER APPLY` against, and partly because it models real systems where sensitive fields (salary) are separated for access control. In a greenfield design you might fold them into `Users`; in an enterprise codebase you will meet this split constantly.

Creating it

```

CREATE SCHEMA TutorialAppSchema;
GO

CREATE TABLE TutorialAppSchema.Users (
    UserId INT IDENTITY(1,1) PRIMARY KEY,
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    Email NVARCHAR(50) NOT NULL,
    Gender NVARCHAR(50) NOT NULL,
    Active BIT NOT NULL
);
GO

CREATE TABLE TutorialAppSchema.UserSalary (
    UserId INT PRIMARY KEY,
    Salary DECIMAL(18,4) NOT NULL
);
GO

CREATE TABLE TutorialAppSchema.UserJobInfo (
    UserId INT PRIMARY KEY,
    JobTitle NVARCHAR(50) NOT NULL,
    Department NVARCHAR(50) NOT NULL
);
GO

CREATE TABLE TutorialAppSchema.Auth (
    Email NVARCHAR(50) NOT NULL PRIMARY KEY,
    PasswordHash VARBINARY(MAX) NOT NULL,
    PasswordSalt VARBINARY(MAX) NOT NULL
);
GO

CREATE TABLE TutorialAppSchema.Posts (
    PostId INT IDENTITY(1,1) PRIMARY KEY,
    UserId INT NOT NULL,
    PostTitle NVARCHAR(255) NOT NULL,
    PostContent NVARCHAR(MAX) NOT NULL,
    PostCreated DATETIME NOT NULL,
    PostUpdated DATETIME NOT NULL
);
GO

```

Key syntax to absorb:

- **IDENTITY(1,1)** — auto-increment, seed 1, step 1. The T-SQL equivalent of MySQL's **AUTO_INCREMENT** or Postgres's **SERIAL**.
- **NVARCHAR vs VARCHAR** — **N** means Unicode (UTF-16, 2 bytes per character). **VARCHAR** is single-byte. **Use NVARCHAR**. Your data will contain Arabic names; **VARCHAR** will turn them into question marks. This is not hypothetical for you.
- **BIT** — the SQL Server boolean. 0, 1, or NULL. Maps to C# **bool**.
- **DECIMAL(18,4)** — 18 total digits, 4 after the decimal point. Maps to C# **decimal**.
- **VARBINARY(MAX)** — raw bytes, for the password hash and salt. Maps to C# **byte[]**.
- **GO** — not SQL. It is a *batch separator* understood by SSMS, Azure Data Studio and **sqlcmd**, telling the tool to send everything above it as one batch. Some statements (**CREATE SCHEMA**, **CREATE PROCEDURE**) must be the first statement in their batch, which is why **GO** appears between them. **GO will fail if you send it through Dapper or ADO.NET** — it is a tooling keyword, not a database one.

ON THE JOB **IDENTITY** integer keys versus **UNIQUEIDENTIFIER** (GUID) keys is a real architectural argument you will hear. Identity integers are compact, index well and are human-readable; GUIDs are globally unique, can be generated client-side without a round trip, and do not leak row counts to an attacker. Distributed systems tend toward GUIDs; single-database line-of-business apps toward identity ints. Have an opinion but defer to the existing codebase.

2.2 SELECT (L28 SQL Basics Pt 1 — 27 min)

```

SELECT * FROM TutorialAppSchema.Users;

SELECT UserId, FirstName, LastName
FROM TutorialAppSchema.Users;

SELECT FirstName + ' ' + LastName AS FullName    -- + concatenates in T-SQL
FROM TutorialAppSchema.Users;

SELECT DISTINCT Department FROM TutorialAppSchema.UserJobInfo;

SELECT TOP 10 * FROM TutorialAppSchema.Users;    -- T-SQL uses TOP, not LIMIT

```

TRAP **SELECT *** is fine when exploring interactively. It is **not** fine in application code or a stored procedure: it breaks the moment someone adds a column, it pulls data over the wire you do not need, and it prevents covering indexes from being used. Name your columns. Reviewers will flag this.

WHERE

```

SELECT * FROM TutorialAppSchema.Users WHERE Active = 1;

SELECT * FROM TutorialAppSchema.Users
WHERE Active = 1 AND Gender = 'Female';

SELECT * FROM TutorialAppSchema.Users
WHERE Department IN ('Engineering', 'Finance');

SELECT * FROM TutorialAppSchema.UserSalary
WHERE Salary BETWEEN 50000 AND 100000;

SELECT * FROM TutorialAppSchema.Users
WHERE Email LIKE '%@pwc.com';      -- % = any characters, _ = one character

SELECT * FROM TutorialAppSchema.Users
WHERE LastName IS NULL;           -- IS NULL, never = NULL

```

TRAP — NULL comparison `NULL` is not a value; it is the absence of one. `WHERE column = NULL` is never true, not even for `NULL` rows, because `NULL = NULL` evaluates to `UNKNOWN`. Always use `IS NULL` and `IS NOT NULL`.

The same trap bites inside `NOT IN`: if the subquery returns any `NULL`, `NOT IN` returns no rows at all. Use `NOT EXISTS` instead.

ORDER BY, and the important limitation on TOP

```

SELECT * FROM TutorialAppSchema.Users ORDER BY LastName ASC;
SELECT * FROM TutorialAppSchema.Users ORDER BY LastName DESC, FirstName ASC;

-- Pagination: OFFSET/FETCH requires ORDER BY
SELECT UserId, FirstName, LastName
FROM TutorialAppSchema.Users
ORDER BY UserId
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;

```

TRAP `SELECT TOP 10` without `ORDER BY` returns an arbitrary ten rows. SQL Server makes no ordering guarantee absent `ORDER BY` — and the rows you get can change between executions as the plan changes. “Top 10” means nothing without “top by what”.

INSERT, UPDATE, DELETE

```

INSERT INTO TutorialAppSchema.Users (FirstName, LastName, Email, Gender, Active)
VALUES ('Yassin', 'Farid', 'yassin@example.com', 'Male', 1);

-- Multi-row insert
INSERT INTO TutorialAppSchema.Users (FirstName, LastName, Email, Gender, Active)
VALUES ('A', 'One', 'a@x.com', 'Male', 1),
      ('B', 'Two', 'b@x.com', 'Female', 1);

UPDATE TutorialAppSchema.Users
SET Active = 0
WHERE UserId = 1;

DELETE FROM TutorialAppSchema.Users WHERE UserId = 1;

```

Note that you do not supply `UserId` on insert — `IDENTITY` generates it. To retrieve the generated value:

```

INSERT INTO TutorialAppSchema.Users (...) VALUES (...);
SELECT SCOPE_IDENTITY();      -- the ID just generated, in this scope

```

`SCOPE_IDENTITY()` is the correct choice. `@@IDENTITY` returns the last identity generated in the *session* — including one generated by a trigger on another table, which is a classic production bug.

TRAP — the career-limiting one An `UPDATE` or `DELETE` without a `WHERE` clause affects **every row in the table**. Always write the `WHERE` first, or write it as a `SELECT` first and convert it once you have confirmed the row count. On a client’s production database this is the mistake people get sent home over.

2.3 Joins (L29 SQL Basics Pt 2 — 21 min)

```
-- INNER JOIN: only rows present in both
SELECT u.UserId, u.FirstName, u.LastName, s.Salary
FROM TutorialAppSchema.Users AS u
    INNER JOIN TutorialAppSchema.UserSalary AS s
    ON u.UserId = s.UserId;

-- LEFT JOIN: all users, NULL salary where absent
SELECT u.UserId, u.FirstName, u.LastName, s.Salary
FROM TutorialAppSchema.Users AS u
    LEFT JOIN TutorialAppSchema.UserSalary AS s
    ON u.UserId = s.UserId;

-- Multiple joins
SELECT u.UserId, u.FirstName, u.LastName,
    s.Salary, j.JobTitle, j.Department
FROM TutorialAppSchema.Users AS u
    LEFT JOIN TutorialAppSchema.UserSalary AS s ON u.UserId = s.UserId
    LEFT JOIN TutorialAppSchema.UserJobInfo AS j ON u.UserId = j.UserId;
```

Join	Returns
INNER JOIN	Only matching rows from both sides
LEFT JOIN	All left rows; NULLs where right has no match
RIGHT JOIN	All right rows; NULLs where left has no match
FULL OUTER JOIN	All rows from both; NULLs on either side
CROSS JOIN	Cartesian product — every combination

Table aliases (**AS u**) are not decoration. Once you have three joins, unaliased column references become ambiguous and unreadable.

TRAP A missing or wrong **ON** clause silently produces a cartesian product. Ten users joined to ten salaries with a bad **ON** gives you one hundred rows, not ten, and the query still “works”. If a result set is suspiciously large, check your join conditions first.

GROUP BY and aggregates

```
SELECT Department, COUNT(*) AS UserCount
FROM TutorialAppSchema.UserJobInfo
GROUP BY Department;

SELECT j.Department,
    COUNT(*) AS Headcount,
    AVG(s.Salary) AS AvgSalary,
    MAX(s.Salary) AS MaxSalary
FROM TutorialAppSchema.UserJobInfo AS j
    INNER JOIN TutorialAppSchema.UserSalary AS s ON j.UserId = s.UserId
GROUP BY j.Department
HAVING COUNT(*) > 5 -- HAVING filters groups; WHERE filters rows
ORDER BY AvgSalary DESC;
```

WHERE filters rows before grouping. **HAVING** filters groups after. You cannot use an aggregate in **WHERE**; you cannot use it to filter individual rows in **HAVING**. This is a standard interview question.

Every non-aggregated column in the **SELECT** must appear in **GROUP BY**. SQL Server enforces this strictly — unlike MySQL’s historical laxity.

Aggregates ignore NULLs, except **COUNT(*)** which counts rows. **COUNT(column)** counts non-NULL values in that column. The difference matters.

2.4 Intermediate T-SQL (L57 SQL Intermediate — 61 min, plus Quiz 7)

Subqueries

```
-- Scalar subquery
SELECT FirstName, LastName
FROM TutorialAppSchema.Users
WHERE UserId IN (
    SELECT UserId FROM TutorialAppSchema.UserSalary WHERE Salary > 100000
);

-- Correlated subquery – runs per outer row
SELECT u.FirstName,
    (SELECT COUNT(*) FROM TutorialAppSchema.Posts p WHERE p.UserId = u.UserId) AS PostCount
FROM TutorialAppSchema.Users u;

-- EXISTS – usually faster than IN, and NULL-safe
SELECT u.FirstName
FROM TutorialAppSchema.Users u
WHERE EXISTS (SELECT 1 FROM TutorialAppSchema.Posts p WHERE p.UserId = u.UserId);
```

CTEs (Common Table Expressions)

```

WITH HighEarnings AS (
    SELECT UserId, Salary
    FROM TutorialAppSchema.UserSalary
    WHERE Salary > 100000
)
SELECT u.FirstName, u.LastName, h.Salary
FROM TutorialAppSchema.Users u
    INNER JOIN HighEarnings h ON u.UserId = h.UserId;

```

A CTE is a named temporary result set that exists for one statement. It makes complex queries readable by naming intermediate steps, and it is how you write recursive queries (org charts, category trees) in T-SQL.

Window functions

```

SELECT
    u.FirstName,
    j.Department,
    s.Salary,
    ROW_NUMBER() OVER (PARTITION BY j.Department ORDER BY s.Salary DESC) AS RankInDept,
    AVG(s.Salary) OVER (PARTITION BY j.Department) AS DeptAverage,
    s.Salary - AVG(s.Salary) OVER (PARTITION BY j.Department) AS DiffFromAvg
FROM TutorialAppSchema.Users u
    INNER JOIN TutorialAppSchema.UserJobInfo j ON u.UserId = j.UserId
    INNER JOIN TutorialAppSchema.UserSalary s ON u.UserId = s.UserId;

```

The difference from `GROUP BY`: window functions compute an aggregate **without collapsing rows**. You keep every row and attach the group-level value to it. `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `LAG()`, `LEAD()`, `SUM() OVER`, `AVG() OVER`.

ON THE JOB Window functions are the single most useful intermediate SQL skill for reporting and analytics work — which is a large share of what PwC builds. “Top 3 per category”, “running total”, “compare each row to its group average”, “difference from previous month” are all one window function and are all genuinely hard without them. Given your Power BI and data background, these will feel natural.

CASE

```

SELECT FirstName,
    CASE
        WHEN Salary > 100000 THEN 'Senior'
        WHEN Salary > 60000 THEN 'Mid'
        ELSE 'Junior'
    END AS Band
FROM ...;

```

NULL handling functions

```

SELECT ISNULL(MiddleName, '') AS MiddleName, -- T-SQL specific, 2 args
    COALESCE(Nickname, FirstName, '') AS Display, -- ANSI, N args
    NULLIF(Discount, 0) AS Discount -- NULL if equal
FROM ...;

```

String, date and conversion functions

```

-- Strings
LEN(s), UPPER(s), LOWER(s), TRIM(s), LTRIM(s), RTRIM(s)
SUBSTRING(s, start, length) -- 1-indexed, not 0
CHARINDEX('@', Email) -- position, 0 if absent
REPLACE(s, 'old', 'new')
CONCAT(a, b, c) -- NULL-safe, unlike +
STRING_AGG(FirstName, ', ') -- group concatenation

-- Dates
GETDATE() -- current local datetime
GETUTCDATE() -- current UTC
SYSDATETIME() -- higher precision
DATEADD(DAY, 7, GETDATE())
DATEDIFF(DAY, StartDate, EndDate)
DATEPART(YEAR, SomeDate)
EOMONTH(GETDATE()) -- end of month
FORMAT(SomeDate, 'yyyy-MM-dd') -- convenient but SLOW at scale

-- Conversion
CAST(SomeValue AS INT)
CONVERT(NVARCHAR(10), SomeDate, 120) -- 120 = ODBC canonical yyyy-mm-dd
TRY_CAST(SomeValue AS INT) -- NULL instead of an error
TRY_CONVERT(INT, SomeValue)

```

TRAP — L39 “Error converting varchar to numeric” The course has a whole lecture note on this error because it is so common. It means SQL Server tried to implicitly convert a text value to a number and failed — usually because a column is `NVARCHAR` when it should be numeric, or because a parameter arrived as a string. Use `TRY_CAST` / `TRY_CONVERT` to get `NULL` instead of a thrown error when you are investigating.

Indexes

```
CREATE INDEX IX_Users_Email ON TutorialAppSchema.Users(Email);
CREATE UNIQUE INDEX IX_Users_Email_Unique ON TutorialAppSchema.Users(Email);
CREATE INDEX IX_Posts_UserId ON TutorialAppSchema.Posts(UserId) INCLUDE (PostTitle);
```

A **clustered index** determines the physical row order — one per table, and the primary key gets it by default. **Non-clustered indexes** are separate structures pointing back at the rows.

Index foreign keys and any column you filter or join on frequently. Do not index everything: every index must be maintained on every insert, update and delete, so they trade write speed for read speed.

TRAP Wrapping a column in a function in your `WHERE` clause makes the index unusable — the query becomes “non-sargable”. `WHERE YEAR(CreatedDate) = 2026` scans the whole table. `WHERE CreatedDate >= '2026-01-01' AND CreatedDate < '2027-01-01'` uses the index. This is one of the highest-impact performance fixes you can make and a good thing to spot in a code review.

Transactions

```
BEGIN TRANSACTION;
BEGIN TRY
    UPDATE TutorialAppSchema.Users SET Active = 0 WHERE UserId = 1;
    DELETE FROM TutorialAppSchema.UserSalary WHERE UserId = 1;
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    THROW; -- re-raise, preserving the original error
END CATCH
```

ACID: Atomicity, Consistency, Isolation, Durability. Any multi-statement operation that must succeed or fail as a unit belongs in a transaction. `THROW` (rather than the older `RAISERROR`) re-raises with the original error number and message intact.

2.5 The seed script (L54 Skipping Crash Course, L55 MacOS and Linux Users, L56 SQL Seed)

Section 5 is four minutes long and exists for one reason: Dominic provides a script that populates the database with realistic test data so that people skipping the SQL track can still follow the API track. Download it from the lecture resources and run it.

ON THE JOB Seeding matters more than the four-minute runtime suggests. Every real project needs a repeatable way to get a developer from a clean clone to a working local database. On your project, look for this on day one: it will be an SQL script, an EF Core migration with a `HasData` seed, or a Docker Compose setup. If there is not one, creating it is an outstanding first contribution — high value, low risk, and it makes you the person who fixed onboarding.

2.6 Connecting from VS Code (L7 MS SQL Extension, L37 Database Connections, L38 Windows Connection Troubleshooting)

A **connection string** tells your application where the database is and how to authenticate:

```
Server=localhost;Database=DotNetCourseDatabase;Trusted_Connection=true;TrustServerCertificate=true;
```

Part	Meaning
<code>Server</code>	Host, optionally <code>host\instance</code> or <code>host,port</code>
<code>Database</code>	Initial catalog
<code>Trusted_Connection=true</code>	Windows Authentication — use the logged-in Windows account
<code>User Id= / Password=</code>	SQL Server Authentication
<code>TrustServerCertificate=true</code>	Accept a self-signed dev certificate
<code>Encrypt=true</code>	Encrypt in transit — mandatory for Azure SQL

Connection troubleshooting, in the order the course suggests:

1. Is the SQL Server service actually running? (`services.msc` on Windows, `docker ps` on Mac/Linux.)
2. Is TCP/IP enabled in SQL Server Configuration Manager? It is off by default in some installs.

3. Is the port (1433) open through the firewall?
4. Is the instance name right? A named instance needs `localhost\SQLEXPRESS`.
5. Missing `TrustServerCertificate=true` produces a certificate-chain error on a fresh local install.

TRAP — the one that ends careers early Never commit a connection string containing a password. `appsettings.json` is in source control. Module 3 covers User Secrets, which is the correct local answer; Module 8 covers Azure App Service configuration and Key Vault, which is the correct deployed answer. A password in a public repository is a security incident, and at a Big Four firm it is a reportable one. If you ever do it by accident, rotate the credential immediately and tell someone — do not quietly force-push.

2.7 T-SQL versus other dialects

If your SQL was learned on PostgreSQL or MySQL:

Concept	T-SQL	PostgreSQL / MySQL
Row limiting	<code>SELECT TOP 10</code>	<code>LIMIT 10</code>
Auto-increment	<code>IDENTITY(1,1)</code>	<code>SERIAL / AUTO_INCREMENT</code>
String concat	<code>+</code> or <code>CONCAT()</code>	<code>\ \ </code> or <code>CONCAT()</code>
Current time	<code>GETDATE()</code>	<code>NOW()</code>
NULL default	<code>ISNULL(a, b)</code>	<code>COALESCE(a, b)</code>
Temp table	<code>#TempName</code>	<code>TEMP TABLE</code>
Variable	<code>DECLARE @x INT</code>	<code>DECLARE x INT</code>
Batch separator	<code>GO</code>	—
Identifier quoting	<code>[Square Brackets]</code>	<code>"Double quotes" / backticks</code>
Pagination	<code>OFFSET .. FETCH NEXT</code>	<code>LIMIT .. OFFSET</code>

Practice — Module 2

Replaces Quiz 4 and Quiz 7. Run these against the seeded course database.

1. **Schema.** Write, from memory, the `CREATE TABLE` for `TutorialAppSchema.Users` with correct types, the identity primary key and appropriate nullability. Explain in a comment why `NVARCHAR` rather than `VARCHAR`.
2. **Basic SELECT.** Return the ten most recently created posts with the author's full name as a single `AuthorName` column.
3. **NULL handling.** Find all users with no salary record. Write it twice: once with `LEFT JOIN ... IS NULL`, once with `NOT EXISTS`. Say which you prefer and why.
4. **Joins.** Return every user with their job title, department and salary, including users missing one or both of those records. Then change it so only users having both appear, and name the one keyword you changed.
5. **Aggregation.** Per department: headcount, average salary, highest salary. Show only departments with more than three people, ordered by average salary descending.
6. **WHERE vs HAVING.** Take the previous query and additionally exclude inactive users. State which clause that filter belongs in and why it cannot go in the other.
7. **CTE.** Using a CTE, find users whose salary is above their own department's average.
8. **Window function.** Rank users by salary within each department and return only the top three per department. Then write the same thing without a window function and note how much harder it is.
9. **CASE.** Add a `Band` column — Senior above 100k, Mid above 60k, Junior otherwise — to a user-and-salary query.
10. **Dates.** Return all posts created in the last thirty days. Write it once with `DATEDIFF` on the column and once with a date range, and say which one can use an index.
11. **Insert and identity.** Insert a user and return the generated `UserId` in the same batch. Use the correct function and say why the alternative is unsafe.
12. **Transaction.** Write a transaction that deactivates a user and deletes their salary row, rolling back cleanly on error.
13. **Index.** Write an index that would speed up looking up a user by email, and one that would help fetching a user's posts. Explain what makes a query non-sargable.
14. **The dangerous one.** Write an `UPDATE` that deactivates one user. Then write down the exact habit you will follow to make sure you never run it without the `WHERE`.

Self-check — Module 2

1. What is a schema in SQL Server, and why does the course use `TutorialAppSchema` rather than `dbo`?
2. Why is `Auth` a separate table from `Users`?
3. What does `IDENTITY(1,1)` do?
4. `NVARCHAR` or `VARCHAR` — which, and what specifically breaks if you choose wrong?
5. What is `GO`, and what happens if you send it through Dapper?
6. Why does `WHERE column = NULL` return nothing?
7. What is the difference between `WHERE` and `HAVING`?
8. What does `COUNT(*)` count that `COUNT(column)` does not?
9. What does `LEFT JOIN` return that `INNER JOIN` does not?
10. What happens if you write a join with no `ON` clause?
11. Why does `SELECT TOP 10` without `ORDER BY` mean nothing?
12. What is the difference between `SCOPE_IDENTITY()` and `@@IDENTITY`, and why does it matter?
13. What does a window function do that `GROUP BY` cannot?
14. Give an example of a non-sargable `WHERE` clause and its sargable rewrite.
15. What do the four letters of ACID stand for?
16. Why should `THROW` be preferred to swallowing an error in a `CATCH` block?
17. Name three things to check when a local SQL Server connection fails.
18. What is `TrustServerCertificate=true` for, and why must you not use it in production?
19. Where should a database password live, if not in `appsettings.json`?
20. Give the T-SQL equivalent of `LIMIT 10`, `NOW()` and `SERIAL`.

Module 3 — C# Intermediate: Classes, Data Access, Config, JSON and Async

Course section 4. Lectures 30–53, plus Quiz 5 (Dapper Definitions) and Quiz 6 (Dynamic Classes and Methods). 4h 12m of video — the densest section in the course and the one that makes everything after it possible.

3.1 Classes, objects and properties (L30 Intermediate Theory, L31 Models — 16 min)

```
public class User
{
    // Auto-implemented properties – the compiler generates the backing field
    public int    UserId    { get; set; }
    public string FirstName { get; set; }
    public string LastName  { get; set; }
    public string Email     { get; set; }
    public string Gender    { get; set; }
    public bool   Active    { get; set; }
}
```

Properties are the defining idiom of C#. `{ get; set; }` generates a hidden private field plus a getter and setter. There is no need for the `getFirstName()` / `setFirstName()` boilerplate Java requires, and unlike a Java public field, a property can later gain validation logic without changing any calling code.

```
// Read-only from outside, settable in the constructor
public int UserId { get; private set; }

// Computed – no backing field at all
public string FullName => $"{FirstName} {LastName}";

// Full property with a backing field, when you need logic
private string _email = string.Empty;
public string Email
{
    get => _email;
    set => _email = value?.Trim().ToLowerInvariant() ?? string.Empty;
}

// init – settable only during object initialisation, immutable after
public int UserId { get; init; }

// required – the compiler forces the caller to set it
public required string Email { get; set; }
```

JAVA BRIDGE A C# property is a Java getter/setter pair collapsed into field-like syntax. `user.FirstName = "Yassin"` calls the setter; it is not direct field access. Java records are closest to C# `record` types; Lombok's `@Data` is what C# properties give you natively.

Object initialiser syntax

```
User user = new User
{
    FirstName = "Yassin",
    LastName  = "Farid",
    Email     = "yassin@example.com",
    Active    = true
};

// Target-typed new – the type is already known from the left side
User user = new() { FirstName = "Yassin" };
```

String property defaults (L32 — a 1-minute note, but it prevents a real annoyance)

With nullable reference types enabled, this warns:

```
public class User
{
    public string FirstName { get; set; } // ⚠ non-nullable, never initialised
}
```

Four legitimate fixes:

```
public string FirstName { get; set; } = string.Empty; // default to empty – course's choice
public string? FirstName { get; set; }                // genuinely optional
public required string FirstName { get; set; }         // caller must supply it
public User(string firstName) => FirstName = firstName; // constructor
```

Dominic uses `= string.Empty` throughout, which keeps models simple and dodges null checks everywhere downstream. It is a reasonable convention for DTOs and database models. For domain entities where “not set” is meaningfully different from “empty”, prefer `required` or `?`.

Constructors, inheritance, interfaces

```
public class User
{
    public User() { } // parameterless – needed by many ORMs
    public User(string firstName, string lastName)
    {
        FirstName = firstName;
        LastName = lastName;
    }
}

public class Employee : User // ':' means both extends AND implements
{
    public string JobTitle { get; set; } = string.Empty;

    public Employee(string firstName, string lastName)
        : base(firstName, lastName) { } // base, not super

    public override string ToString() => $"{FullName} – {JobTitle}";
}

public interface IUserRepository
{
    IEnumerable<User> GetUsers();
    User GetUserById(int userId);
    bool UpsertUser(User user);
}

public class UserRepository : IUserRepository { /* ... */ }
```

Differences from Java worth memorising:

- `:` serves as both `extends` and `implements`.
- `base` rather than `super`.
- **virtual / override are explicit.** A method is *not* overridable unless the base marks it `virtual` (or `abstract`). Java is the opposite — everything is virtual by default. Forgetting `virtual` and then trying to `override` is a common early error.
- `sealed` = Java’s `final` for classes and methods.
- Single class inheritance, multiple interface implementation — same as Java.

record types

```
public record UserDto(int UserId, string FirstName, string LastName);

// Value equality for free
var a = new UserDto(1, "Yassin", "Farid");
var b = new UserDto(1, "Yassin", "Farid");
bool same = a == b; // true – compares values, not references

// Non-destructive mutation
var updated = a with { LastName = "Hatem" };
```

Records give you value equality, a generated constructor, `ToString()`, and deconstruction. Ideal for DTOs. Java records are the direct analogue.

3.2 Namespaces (L33 Namespaces — 5 min)

```
// File-scoped namespace (C# 10+) – one per file, no braces, no indentation
namespace DotnetAPI.Models;

public class User { }
```

```
// Classic block form – still valid, common in older code
namespace DotnetAPI.Models
{
    public class User { }
}
```

Using them:

```
using System;
using System.Collections.Generic;
using DotnetAPI.Models;
using Models = DotnetAPI.Models; // alias, for name collisions
global using DotnetAPI.Models; // applies to the whole project – put in GlobalUsings.cs
```

JAVA BRIDGE Namespaces are Java packages, with one crucial difference: **a namespace does not have to match the folder structure**. C# does not enforce it. Convention still says match it — `DotnetAPI/Models/User.cs` should be `namespace DotnetAPI.Models` — and every team enforces it by convention.

`ImplicitUsings` in the `.csproj` auto-imports the common namespaces, which is why new files do not start with a wall of `using` lines.

3.3 Dapper (L34 Dapper Definitions and Planning, L40 Dapper Pt 1 — 18 min, L41 Dapper Pt 2 — 20 min, Quiz 5, Quiz 6)

Dapper is a micro-ORM. It does one job: execute SQL you wrote and map the result rows onto C# objects. It does not generate SQL, track changes, manage relationships or do migrations. It is a thin, extremely fast layer over ADO.NET.

JAVA BRIDGE Dapper $\approx$ Spring's `JdbcTemplate` with automatic row mapping. Entity Framework Core $\approx$ Hibernate/JPA. You have used both patterns in Spring Boot; the trade-off is identical.

Setup

```
dotnet add package Dapper
dotnet add package Microsoft.Data.SqlClient
```

The DataContextDapper class the course builds

The course wraps Dapper in a small helper class so that the connection string lives in one place:

```
using System.Data;
using Dapper;
using Microsoft.Data.SqlClient;

namespace DotnetAPI.Data;

public class DataContextDapper
{
    private readonly IConfiguration _config;

    public DataContextDapper(IConfiguration config)
    {
        _config = config;
    }

    public IEnumerable<T> LoadData<T>(string sql)
    {
        IDbConnection dbConnection =
            new SqlConnection(_config.GetConnectionString("DefaultConnection"));
        return dbConnection.Query<T>(sql);
    }

    public T LoadDataSingle<T>(string sql)
    {
        IDbConnection dbConnection =
            new SqlConnection(_config.GetConnectionString("DefaultConnection"));
        return dbConnection.QuerySingle<T>(sql);
    }

    public bool ExecuteSql(string sql)
    {
        IDbConnection dbConnection =
            new SqlConnection(_config.GetConnectionString("DefaultConnection"));
        return dbConnection.Execute(sql) > 0;
    }

    public int ExecuteSqlWithRowCount(string sql)
    {
        IDbConnection dbConnection =
            new SqlConnection(_config.GetConnectionString("DefaultConnection"));
        return dbConnection.Execute(sql);
    }
}
```

Note that `IDbConnection` is an interface and Dapper's methods are extension methods on it — that is why `using Dapper;` is required for `Query<T>` to appear at all. Note also that the connection is not explicitly opened or closed: Dapper opens it if closed and returns it to the ADO.NET connection pool when done.

The core Dapper API

```
// Many rows
IEnumerable<User> users = connection.Query<User>("SELECT * FROM TutorialAppSchema.Users");

// Exactly one – throws if zero or more than one
User user = connection.QuerySingle<User>(sql);
User? user = connection.QuerySingleOrDefault<User>(sql); // null if zero

// First row
User user = connection.QueryFirst<User>(sql);
User? user = connection.QueryFirstOrDefault<User>(sql);

// Non-query: returns rows affected
int rows = connection.Execute("UPDATE ... SET ...");

// Single value
int count = connection.ExecuteScalar<int>("SELECT COUNT(*) FROM ...");

// Async versions of all of the above
var users = await connection.QueryAsync<User>(sql);
```

Mapping is by **name match** between result columns and the type's property names, case-insensitively. A column named `FirstName` fills a property named `FirstName`. Anything unmatched is silently ignored in both directions — which is convenient and occasionally maddening.

TRAP If a property is always null after a query, the cause is almost always a name mismatch — an aliasing difference, a typo, or a column named `first_name` against a property named `FirstName`. Alias in SQL: `SELECT first_name AS FirstName`. Dapper will not tell you it failed to match; it just leaves the default value.

Parameters — and the security point that matters most in this entire course

Never build SQL by string concatenation with user input.

```
// ✗ SQL INJECTION – never do this
string sql = "SELECT * FROM Users WHERE Email = '" + email + "'";
```

If `email` is `' OR '1'='1' --`, that query returns every user. If it is `'; DROP TABLE Users; --`, you have lost the table. This is the number-one application security vulnerability in the world and it is trivially preventable.

```
// ✓ Parameterised – Dapper sends the value separately from the SQL text
string sql = "SELECT * FROM TutorialAppSchema.Users WHERE Email = @Email";
var users = connection.Query<User>(sql, new { Email = email });
```

The `new { Email = email }` is an **anonymous object**; Dapper turns each property into a `SqlParameter`. The database receives the query text and the values as separate things, so a value can never be parsed as SQL. It is also faster, because SQL Server caches the execution plan for the parameterised query text.

ON THE JOB SQL injection will appear in any security review, penetration test or client audit of your project. At a Big Four firm, shipping a concatenated query into a client's system is a serious finding. Parameterise everything, always, even for values you believe are safe — because “this value comes from our own code” stops being true the moment someone refactors.

DynamicParameters (the basis for lecture 124)

For parameters built conditionally at runtime:

```
DynamicParameters sqlParameters = new DynamicParameters();
sqlParameters.Add("@UserIdParameter", userId, DbType.Int32);
sqlParameters.Add("@ActiveParameter", active, DbType.Boolean);

var users = connection.Query<User>(sql, sqlParameters);
```

Section 10 builds an entire dynamic-query system on this, so make sure the concept lands: `DynamicParameters` is a bag you add to imperatively, rather than an anonymous object you declare all at once.

Multi-mapping — joining into object graphs

```
string sql = @"
SELECT u.UserId, u.FirstName, u.LastName, s.UserId, s.Salary
FROM TutorialAppSchema.Users u
JOIN TutorialAppSchema.UserSalary s ON u.UserId = s.UserId";

var users = connection.Query<User, UserSalary, User>(
    sql,
    (user, salary) => { user.Salary = salary; return user; },
    splitOn: "UserId" // where the second object's columns begin
);
```

`splitOn` tells Dapper which column starts each subsequent object. Getting it wrong produces nulls in the nested object — another common silent failure.

Calling a stored procedure

```
var users = connection.Query<User>(  
    "TutorialAppSchema.spUsers_Get",  
    new { UserId = userId },  
    commandType: CommandType.StoredProcedure  
);
```

Module 6 covers this properly.

3.4 Entity Framework Core (L42 EF Definitions and Planning — 9 min, L43 Entity Framework — 28 min)

EF Core is a **full ORM**: it generates SQL for you, tracks changes to loaded objects, manages relationships, and handles schema migrations.

```
dotnet add package Microsoft.EntityFrameworkCore  
dotnet add package Microsoft.EntityFrameworkCore.SqlServer  
dotnet add package Microsoft.EntityFrameworkCore.Design  
dotnet tool install --global dotnet-ef
```

The DbContext

```
using Microsoft.EntityFrameworkCore;  
using DotnetAPI.Models;  
  
namespace DotnetAPI.Data;  
  
public class DataContextEF : DbContext  
{  
    private readonly IConfiguration _config;  
  
    public DataContextEF(IConfiguration config)  
    {  
        _config = config;  
    }  
  
    public virtual DbSet<User> Users { get; set; }  
    public virtual DbSet<UserSalary> UserSalary { get; set; }  
    public virtual DbSet<UserJobInfo> UserJobInfo { get; set; }  
    public virtual DbSet<Auth> Auth { get; set; }  
    public virtual DbSet<Post> Posts { get; set; }  
  
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)  
    {  
        if (!optionsBuilder.IsConfigured)  
        {  
            optionsBuilder.UseSqlServer(  
                _config.GetConnectionString("DefaultConnection"),  
                options => options.EnableRetryOnFailure()  
            );  
        }  
    }  
  
    protected override void OnModelCreating(ModelBuilder modelBuilder)  
    {  
        modelBuilder.HasDefaultSchema("TutorialAppSchema");  
  
        modelBuilder.Entity<User>()  
            .ToTable("Users", "TutorialAppSchema")  
            .HasKey(u => u.UserId);  
  
        modelBuilder.Entity<UserSalary>()  
            .ToTable("UserSalary", "TutorialAppSchema")  
            .HasKey(u => u.UserId);  
  
        modelBuilder.Entity<Auth>()  
            .HasKey(a => a.Email);  
    }  
}
```

`DbSet<T>` is a queryable collection representing a table. `OnModelCreating` is the **Fluent API**, where you configure mappings that convention cannot infer — a non-default schema, a composite key, a column name that differs from the property name.

JAVA BRIDGE `DbContext` $\approx$ JPA's `EntityManager` / Hibernate `Session`. `DbSet<T>` $\approx$ a JPA repository or `EntityManager` typed query root. `OnModelCreating` fluent configuration $\approx$ JPA annotations or `orm.xml`. `SaveChanges()` $\approx$ `EntityManager.flush()` inside a transaction.

CRUD with EF Core

```
// Read – LINQ, translated to SQL
IEnumerable<User> users = _entityFramework.Users.ToList();

User? user = _entityFramework.Users
    .Where(u => u.UserId == userId)
    .FirstOrDefault();

var active = _entityFramework.Users
    .Where(u => u.Active)
    .OrderBy(u => u.LastName)
    .ToList();

// Create
_entityFramework.Add(user);
_entityFramework.SaveChanges();

// Update – change tracking notices the mutation
User? existing = _entityFramework.Users.FirstOrDefault(u => u.UserId == id);
if (existing != null)
{
    existing.FirstName = "NewName";
    _entityFramework.SaveChanges();    // EF generates the UPDATE
}

// Delete
_entityFramework.Remove(user);
_entityFramework.SaveChanges();

// Related data
var withPosts = _entityFramework.Users
    .Include(u => u.Posts)                // eager load – avoids N+1
    .ToList();

// Read-only query – skip change tracking for speed
var readOnly = _entityFramework.Users.AsNoTracking().ToList();
```

Change tracking is EF’s defining feature and the source of most surprise. When you load an entity, EF keeps a snapshot. `SaveChanges()` diffs current state against the snapshot and generates exactly the required **INSERT** / **UPDATE** / **DELETE** statements, inside a transaction.

TRAP — N+1 queries

```
var users = _ef.Users.ToList();                // 1 query
foreach (var u in users) { var n = u.Posts.Count; } // 1 query PER USER
```

A hundred users become a hundred and one round trips. Fix it with `.Include(u => u.Posts)`. This is the most common EF Core performance bug in existence, it looks harmless in code review, and it only shows up under production data volumes. If you learn one EF lesson, learn this one.

TRAP Forgetting `SaveChanges()` means nothing persists and nothing complains. A “the update didn’t save” bug is this, nine times out of ten.

Migrations

```
dotnet ef migrations add InitialCreate
dotnet ef database update
dotnet ef migrations remove
dotnet ef migrations script                # generate SQL for a DBA to review
```

Migrations are versioned, code-defined schema changes — Flyway or Liquibase, but generated from your model classes.

ON THE JOB Whether migrations run automatically on deploy is a genuine team decision with real consequences. Many enterprise clients forbid it: the application account has no DDL rights, and schema changes go through a DBA as reviewed SQL. `dotnet ef migrations script` produces exactly that SQL. **Ask how schema changes are deployed before you create your first migration.** Getting this wrong in a regulated client environment is a change-control incident, not a coding mistake.

Dapper vs EF Core — the decision you will be asked about

	Dapper	EF Core
SQL	You write it	Generated from LINQ
Speed	Near raw ADO.NET	~10-30% slower typically
Change tracking	None	Yes
Migrations	No	Yes
Relationships	Manual multi-mapping	Navigation properties, <code>Include</code>
Learning curve	Shallow	Steep — LINQ translation, tracking, loading
Control over SQL	Total	Limited; can drop to raw SQL
Best for	Complex queries, reporting, performance-critical paths, stored-procedure-heavy shops	CRUD-heavy business apps, greenfield, rapid delivery

They coexist. A common and entirely respectable architecture uses EF Core for CRUD and Dapper for the handful of complex reporting queries where you want the SQL by hand. This course builds both side by side for exactly that reason — and lectures 81 and 82 solve the same assignment once each way so you can compare them directly.

3.5 Configuration (L44 Config — 18 min)

`appsettings.json`:

```
{
  "ConnectionsStrings": {
    "DefaultConnection": "Server=localhost;Database=DotNetCourseDatabase;Trusted_Connection=true;TrustServerCertificate=true;"
  },
  "AppSettings": {
    "TokenKey": "REPLACE_ME",
    "PasswordKey": "REPLACE_ME"
  },
  "Logging": {
    "LogLevel": { "Default": "Information" }
  }
}
```

Reading it:

```
public class SomeService
{
    private readonly IConfiguration _config;
    public SomeService(IConfiguration config) => _config = config;

    public void Demo()
    {
        string? cs = _config.GetConnectionString("DefaultConnection");
        string? key = _config.GetSection("AppSettings:TokenKey").Value;
        string? nested = _config["AppSettings:PasswordKey"]; // indexer shorthand
        int retries = _config.GetValue<int>("AppSettings:MaxRetries");
    }
}
```

TRAP — L92 “.NET 8 Note: IConfiguration.GetSection()” The course has a dedicated note because this changed between versions. `GetSection("A:B")` returns an `IConfigurationSection` — an object, never null. `.Value` on it may be null. `_config["A:B"]` returns the `string?` directly. Mixing them up produces a confusing null. Use the colon as the nesting separator in both forms.

Configuration layering and precedence

Configuration sources are layered, later overriding earlier:

1. `appsettings.json`
2. `appsettings.{Environment}.json` — e.g. `appsettings.Development.json`
3. **User Secrets** (Development only)
4. Environment variables
5. Command-line arguments

The environment comes from `ASPNETCORE_ENVIRONMENT`: `Development`, `Staging`, or `Production`.

User Secrets — the correct local answer for passwords

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionsStrings:DefaultConnection" "Server=...;Password=..."
dotnet user-secrets list
```

Secrets are stored **outside the project folder**, in your user profile, so they cannot be committed. No code change is needed — `IConfiguration` picks them up automatically in Development.

ON THE JOB The progression you should be able to state: **User Secrets locally → environment variables or App Service configuration in a deployed environment → Azure Key Vault for anything genuinely sensitive in production.** A `TokenKey` or database password committed to a repository is a security incident at a Big Four firm and a reportable one at many clients. Module 8 covers the Azure side.

Strongly-typed configuration — better than `IConfiguration` everywhere

```
public class AppSettings
{
    public string TokenKey { get; set; } = string.Empty;
    public string PasswordKey { get; set; } = string.Empty;
}

// Program.cs
builder.Services.Configure<AppSettings>(builder.Configuration.GetSection("AppSettings"));

// Inject it
public class TokenService
{
    private readonly AppSettings _settings;
    public TokenService(IOptions<AppSettings> options) => _settings = options.Value;
}
```

This gives compile-time safety and testability instead of magic strings scattered through the codebase. The course uses `IConfiguration` directly for simplicity; `IOptions<T>` is what a well-structured production codebase uses, and knowing the difference is worth mentioning if you are asked.

3.6 File read and write (L46 — 14 min)

```
// Read
string text = File.ReadAllText("path.txt");
string[] lines = File.ReadAllLines("path.txt");
byte[] bytes = File.ReadAllBytes("path.bin");

// Write
File.WriteAllText("path.txt", content);
File.AppendAllText("log.txt", line + Environment.NewLine);

// Async — preferred for I/O
string text = await File.ReadAllTextAsync("path.txt");
await File.WriteAllTextAsync("path.txt", content);

// Existence and paths
if (File.Exists(path)) { }
Directory.CreateDirectory(folder);
string full = Path.Combine(folder, "file.txt"); // OS-correct separator
string ext = Path.GetExtension(path);
```

TRAP Never build a path with string concatenation and `"\"`. `Path.Combine` produces the right separator on Windows, Linux and macOS. Your dev laptop is Windows; the Azure App Service your code deploys to is Linux. Hardcoded backslashes break there and nowhere else, which makes them miserable to debug.

`using` for deterministic cleanup:

```
using (StreamReader reader = new StreamReader("file.txt"))
{
    string? line = reader.ReadLine();
} // Dispose() called here, even on exception

// C# 8 using declaration — disposed at end of enclosing scope
using StreamReader reader = new StreamReader("file.txt");
```

JAVA BRIDGE `using` is try-with-resources. `IDisposable` is `AutoCloseable`. Anything holding an unmanaged resource — file handles, network sockets, database connections — should be in a `using`.

3.7 JSON (L47 JSON Definitions and Planning — 5 min, L48 JSON — 31 min)

JSON is how your API talks to the React front end, so this is not an aside.

```

using System.Text.Json;
using System.Text.Json.Serialization;

// Serialise
string json = JsonSerializer.Serialize(user);

// Deserialise
User? user = JsonSerializer.Deserialize<User>(json);
List<User>? users = JsonSerializer.Deserialize<List<User>>(json);

// Options
var options = new JsonSerializerOptions
{
    PropertyNameCaseInsensitive = true,
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    WriteIndented = true,
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull
};
string json = JsonSerializer.Serialize(user, options);

```

Attributes that control the shape:

```

public class User
{
    [JsonPropertyName("user_id")]
    public int UserId { get; set; }

    [JsonIgnore]
    public string PasswordHash { get; set; } = string.Empty;

    [JsonPropertyOrder(1)]
    public string FirstName { get; set; } = string.Empty;
}

```

TRAP — the casing mismatch that will cost you an afternoon on a React project C# properties are PascalCase. JavaScript convention is camelCase. ASP.NET Core’s default JSON serializer **automatically camelCases** on the way out: `FirstName` becomes `firstName` in the response body.

On the way in, `System.Text.Json` is **case-sensitive by default** when you call `JsonSerializer.Deserialize` yourself — though ASP.NET Core’s model binding configures case-insensitivity for you. The result: manual deserialisation silently produces nulls where automatic model binding works fine. When a React developer tells you the API “returns nothing”, check the casing before anything else.

System.Text.Json VS Newtonsoft.Json. `System.Text.Json` is built in, fast, and the default since .NET Core 3.0. `Newtonsoft.Json` (Json.NET) is the older third-party library — more features, more permissive, slower. You will meet both; older projects and some libraries still require Newtonsoft. They have different attribute names (`[JsonProperty]` vs `[JsonPropertyName]`) and different default behaviours, and mixing them in one project causes genuinely confusing bugs.

3.8 AutoMapper and model mapping (L49 AutoMapper Versioning — 1 min, L50 Model Mapping — 29 min)

The problem: you have a `UserToAddDto` from the HTTP request and need a `User` model for the database. Copying six properties by hand, in ten places, is tedious and error-prone.

```

// Manual — explicit, verbose, but no magic
User user = new User
{
    FirstName = userDto.FirstName,
    LastName = userDto.LastName,
    Email = userDto.Email,
    Gender = userDto.Gender,
    Active = userDto.Active
};

```

AutoMapper does it by convention:

```
dotnet add package AutoMapper
```

```

public class AutoMapperProfiles : Profile
{
    public AutoMapperProfiles()
    {
        CreateMap<UserToAddDto, User>();
        CreateMap<User, UserToAddDto>();

        // When names differ
        CreateMap<User, UserDisplayDto>()
            .ForMember(dest => dest.FullName,
                opt => opt.MapFrom(src => src.FirstName + " " + src.LastName));
    }
}

```

```

// Program.cs registration
builder.Services.AddAutoMapper(typeof(AutoMapperProfiles).Assembly);

// Usage
private readonly IMapper _mapper;
User user = _mapper.Map<User>(userDto);

```

TRAP — L49 exists because of this AutoMapper’s registration API has changed repeatedly across major versions, and licensing changed in version 15 (it became commercial for larger organisations). Registration syntax that works in one version fails in another. **Check the installed version before copying any AutoMapper code from a tutorial or from this guide.** `dotnet list package` tells you.

ON THE JOB — have an opinion on this AutoMapper is genuinely contested among senior .NET developers. The case for: eliminates boilerplate, one place to change mappings. The case against: mapping errors become runtime failures rather than compile errors, it is hard to trace, refactoring tools cannot see through it, and `CreateMap` silently ignores properties it cannot match — so a renamed property becomes a silent null in production.

Many teams now prefer manual mapping or source-generated mappers (Mapperly, Mapster) for exactly that reason. Follow your project’s convention, but if you are asked, the mature answer is: “AutoMapper for large, stable, symmetric mappings; manual mapping where correctness matters and the shape differs.” If you do use it,

`_mapper.ConfigurationProvider.AssertConfigurationIsValid()` in a unit test catches the silent-null class of bug.

3.9 Async and await (L51 Async Await Theory — 4 min, L52 Tasks — 12 min, L53 Async Methods — 10 min)

Why it exists

A thread waiting on a database query or an HTTP call is doing nothing but occupying memory. A web server has a finite thread pool. Under load, synchronous I/O exhausts it and requests queue behind threads that are merely *waiting*.

`async/await` releases the thread back to the pool while the I/O is in flight, and resumes the method when it completes. The result is dramatically higher throughput under concurrent load — not faster individual requests.

JAVA BRIDGE Closest to `CompletableFuture`, but the compiler rewrites the method into a state machine so you write ordinary sequential-looking code. Far more ergonomic than Java’s callback chaining. Conceptually similar to Java 21 virtual threads in what it achieves, though the mechanism differs.

Syntax

```

public async Task<User> GetUserAsync(int userId)
{
    User user = await _repository.GetUserByIdAsync(userId);
    return user;
}

public async Task SaveAsync(User user) // Task, not Task<T>, for "void"
{
    await _repository.SaveAsync(user);
}

```

Return type	Meaning
<code>Task<T></code>	Async method returning a <code>T</code>
<code>Task</code>	Async method returning nothing
<code>ValueTask<T></code>	Optimisation for usually-synchronous paths
<code>void</code>	Only for event handlers — never otherwise

```
// Run in parallel and wait for all
Task<User> userTask = GetUserAsync(1);
Task<List<Post>> postsTask = GetPostsAsync(1);
await Task.WhenAll(userTask, postsTask);
User user = await userTask;

// First to finish
await Task.WhenAny(task1, task2);

// Delay – never Thread.Sleep in async code
await Task.Delay(1000);
```

TRAP — deadlock

```
var user = GetUserAsync(1).Result; // ✗ can deadlock
GetUserAsync(1).Wait();           // ✗ same
```

Blocking on an async call can deadlock in contexts with a synchronisation context, and wastes a thread even where it does not. **Async all the way down.** If a method calls something async, it becomes async, and so does its caller, right up to the controller action — which ASP.NET Core handles natively.

TRAP — async void An exception thrown inside an `async void` method cannot be caught by the caller and will crash the process. Use `async Task` for everything except event handlers.

TRAP — the pointless async

```
public async Task<User> GetAsync(int id) => await _repo.GetAsync(id); // extra state machine
public Task<User> GetAsync(int id) => _repo.GetAsync(id);           // just pass it through
```

If you are not doing anything after the `await`, return the task directly. Minor, but reviewers notice.

Naming convention: async methods end in `Async` — `GetUserAsync`, `SaveChangesAsync`. Both Dapper and EF Core provide async versions of everything: `QueryAsync`, `ExecuteAsync`, `ToListAsync`, `FirstOrDefaultAsync`, `SaveChangesAsync`.

ON THE JOB The course writes most of its data access synchronously to keep the teaching focused. **Production ASP.NET Core code should be async throughout.** If you join a project and find synchronous database calls in controllers, that is a genuine scalability observation — raise it as a question rather than a criticism, since there may be history you do not know.

Practice — Module 3

Replaces Quiz 5 (Dapper Definitions) and Quiz 6 (Dynamic Classes and Methods).

- Models.** Write `User`, `UserSalary`, `UserJobInfo` and `Post` classes matching the schema from Module 2 exactly — correct types, correct nullability, correct string defaults. Use `decimal` where the schema says `DECIMAL`.
- Properties.** Add a computed `FullName` to `User` with no backing field. Add an `Email` property whose setter trims and lowercases. Add a `UserId` that is publicly readable but only settable during initialisation.
- Record.** Rewrite `UserDto` as a `record`. Prove value equality with two identical instances, then produce a modified copy with `with`.
- Inheritance.** Make `Employee` inherit from `User`, add `JobTitle`, override `ToString()`. Deliberately omit `virtual` on the base first and record the compiler error.
- Dapper — read.** Write a `DataContextDapper` with `LoadData<T>`, `LoadDataSingle<T>` and `ExecuteSql`, reading the connection string from `IConfiguration`. Query all users.
- Dapper — parameters.** Write a query fetching a user by email. Write the injectable concatenated version first, work out what input would return every row, then rewrite it parameterised. Keep both in comments with a note explaining the attack.
- DynamicParameters.** Write a query that filters by `UserId` only when a supplied `int?` is non-null, building the SQL and the `DynamicParameters` conditionally.
- Dapper — mapping failure.** Deliberately alias a column so it does not match the property name. Observe the silent null. Fix it with a SQL alias.
- EF Core — context.** Write `DataContextEF` with `DbSet`s for all five tables, `OnConfiguring` reading the connection string, and `OnModelCreating` setting the schema and the `Auth` key on `Email`.
- EF Core — CRUD.** Insert a user, read them back, update the last name, delete them. Then deliberately omit `SaveChanges()` on the update and confirm nothing happens and nothing errors.
- N+1.** Write a loop that reads users and then each user's posts, and watch the query count. Rewrite with `.Include()`. Explain in a comment why the first version passes code review but fails in production.

12. **Config.** Move your connection string to `appsettings.json`, read it via `IConfiguration`, then move it into User Secrets and confirm the code needs no change. Add a `TokenKey` under `AppSettings` and read it with both `GetSection("AppSettings:TokenKey").Value` and `_config["AppSettings:TokenKey"]`.
13. **Strongly-typed config.** Bind that `AppSettings` section to a class and inject it with `IOptions<T>`.
14. **File I/O.** Read a text file, transform each line to uppercase, write it back to a new file. Use `Path.Combine`, `File.Exists`, and a `using` declaration. Do it async.
15. **JSON.** Serialise a `User` to JSON. Add `[JsonIgnore]` to a sensitive property and confirm it disappears. Deserialise a camelCase JSON string into your PascalCase class — first without `PropertyNameCaseInsensitive`, observe the nulls, then with it.
16. **AutoMapper.** Create a profile mapping `UserToAddDto` to `User`, register it, and use it. Then add a property to the DTO that has no counterpart and confirm AutoMapper does not complain. Write one sentence on why that is dangerous.
17. **Async.** Rewrite your Dapper `LoadData<T>` as `LoadDataAsync<T>` using `QueryAsync`. Call it with `await`. Then call it with `.Result` and write a comment on why that is wrong.
18. **Task.WhenAll.** Fire two independent async calls in parallel and await both together. Compare the elapsed time against awaiting them one after the other.

Self-check — Module 3

1. What does `{ get; set; }` actually generate, and how does it differ from a public field?
2. When would you use `init` rather than `set`? `required` rather than a default?
3. What does `= string.Empty` on a string property prevent?
4. What does `:` mean in `public class Employee : User, IComparable`?
5. Why does C# require `virtual` on a base method, and how does that differ from Java?
6. What three things does a `record` give you for free?
7. Does a C# namespace have to match the folder path? Should it?
8. In one sentence each: what is Dapper, and what is EF Core?
9. What does Dapper do when a result column has no matching property?
10. Why is `new { Email = email }` safe when string concatenation is not? Explain what the database receives differently.
11. What is `DynamicParameters` for, and when would you choose it over an anonymous object?
12. What does `splitOn` do in Dapper multi-mapping?
13. What is change tracking, and what does `SaveChanges()` actually do?
14. Describe the N+1 problem and its fix.
15. What does `AsNoTracking()` do and when would you use it?
16. Give three reasons a team might forbid running EF migrations automatically on deploy.
17. Name a situation where Dapper is the better choice and one where EF Core is.
18. List the configuration sources in precedence order.
19. Where do User Secrets live, and why does that location matter?
20. What is the difference between `_config["A:B"]` and `_config.GetSection("A:B")`?
21. Why does an API response show `firstName` when the C# property is `FirstName`?
22. What is the practical difference between `System.Text.Json` and `Newtonsoft.Json`?
23. Give the strongest argument for AutoMapper and the strongest argument against it.
24. What problem does `async/await` solve — and what does it *not* make faster?
25. Why is `.Result` on a `Task` dangerous?
26. When is `async void` acceptable?
27. What does `Task.WhenAll` do that sequential `await`s do not?
28. Why must you use `Path.Combine` rather than concatenating with a backslash?

Module 4 — API Basics

Course section 7. Lectures 58–82. 4h 24m. This is where the course stops teaching language features and starts building the actual application.

4.1 REST and API theory (L58 API Basics Theory — 2 min)

An **API** exposes functionality over HTTP. **REST** is an architectural style with a few rules that matter in practice:

- **Resources** are nouns, addressed by URL: `/api/user`, `/api/post/5`.
- **HTTP verbs** express the operation, not the URL.
- **Stateless** — every request carries everything needed to serve it. No server-side session. This is why JWT (Module 5) exists.
- **Status codes** communicate outcome.

Verb	Purpose	Idempotent?	Typical success code
GET	Retrieve	Yes	200 OK
POST	Create	No	201 Created
PUT	Replace entirely	Yes	200 OK / 204 No Content
PATCH	Partial update	No	200 OK
DELETE	Remove	Yes	204 No Content

Idempotent means calling it repeatedly has the same effect as calling it once. `DELETE /api/user/5` five times leaves the same end state. `POST /api/user` five times creates five users. This matters for retry logic — a client can safely retry an idempotent request after a timeout.

Status codes you must use correctly:

Code	Meaning	Use when
200 OK	Success with body	Successful GET, PUT
201 Created	Resource created	Successful POST
204 No Content	Success, empty body	Successful DELETE
400 Bad Request	Client sent invalid data	Validation failure
401 Unauthorized	Not authenticated	Missing or invalid token
403 Forbidden	Authenticated but not permitted	Wrong role
404 Not Found	Resource does not exist	Bad ID
409 Conflict	State conflict	Duplicate email
500 Internal Server Error	Server fault	Unhandled exception

TRAP 401 and 403 are constantly confused. **401 = I do not know who you are.** **403 = I know exactly who you are, and you may not do this.** Getting these right is a small thing that signals you understand HTTP.

4.2 Creating and running the API (L59 Getting Started, L60 Running the API — 20 min, L70 Http Launch)

```
dotnet new webapi -n DotnetAPI
cd DotnetAPI
dotnet run
dotnet watch run           # hot reload — use this while developing
```

Generated structure:

```
DotnetAPI/
├── Controllers/
│   └── WeatherForecastController.cs
├── Properties/
│   └── launchSettings.json
├── appsettings.json
├── appsettings.Development.json
├── Program.cs
└── DotnetAPI.csproj
```

Delete the `WeatherForecast` sample once you have read it — it is a template, not part of your application.

launchSettings.json (L63 — 12 min)

```
{
  "profiles": {
    "http": {
      "commandName": "Project",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "applicationUrl": "https://localhost:5001;http://localhost:5000"
    }
  }
}
```

TRAP `launchSettings.json` is **local development only**. It is not deployed and has no effect in Azure. Developers repeatedly set a configuration value here and wonder why it is missing in production. In deployment, the equivalents are environment variables and App Service configuration (Module 8).

The `ASPNETCORE_ENVIRONMENT` variable here is what makes `appsettings.Development.json` load and what turns Swagger on.

ON THE JOB The HTTPS profile uses a local development certificate. `dotnet dev-certs https --trust` installs it. On a locked-down corporate laptop this sometimes fails and you will need to run the `http` profile locally — worth knowing before you spend an hour on it.

4.3 Program.cs and the pipeline (L61 “Startup.cs” — 15 min)

The lecture is titled `Startup.cs` because that is what it was called before .NET 6. **In .NET 6 and later there is no `Startup.cs`** — its two halves (`ConfigureServices` and `Configure`) were merged into `Program.cs` using the minimal hosting model. You will meet both: any project older than 2021 has `Startup.cs`.

```
var builder = WebApplication.CreateBuilder(args);

// ---- 1. SERVICE REGISTRATION (the old ConfigureServices) ----
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("DevCors", corsBuilder =>
    {
        corsBuilder.WithOrigins("http://localhost:4200",
                                "http://localhost:3000",
                                "http://localhost:8000")
                    .AllowAnyMethod()
                    .AllowAnyHeader()
                    .AllowCredentials();
    });
    options.AddPolicy("ProdCors", corsBuilder =>
    {
        corsBuilder.WithOrigins("https://myproductionsite.com")
                    .AllowAnyMethod()
                    .AllowAnyHeader()
                    .AllowCredentials();
    });
});

builder.Services.AddScoped<UserRepository, UserRepository>();
builder.Services.AddSingleton<DataContextDapper>();
builder.Services.AddDbContext<DataContextEF>();
builder.Services.AddAutoMapper(typeof(Program).Assembly);

var app = builder.Build();

// ---- 2. MIDDLEWARE PIPELINE (the old Configure) ----
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
    app.UseCors("DevCors");
}
else
{
    app.UseCors("ProdCors");
    app.UseHttpsRedirection();
}

app.UseAuthentication(); // who are you? - must come first
app.UseAuthorization(); // may you? - must come second
app.MapControllers();

app.Run();
```

Dependency injection lifetimes

Lifetime	One instance per	Use for
AddTransient	Every injection	Lightweight, stateless services
AddScoped	HTTP request	Repositories, DbContext — the default choice
AddSingleton	Application lifetime	Caches, configuration, stateless helpers

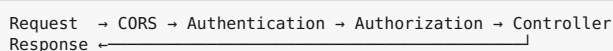
JAVA BRIDGE This is Spring's IoC container with different names. `AddScoped` $\approx$ `@RequestScope`, `AddSingleton` $\approx$ Spring's default singleton `@Service`, `AddTransient` $\approx$ `@Scope("prototype")`. Constructor injection works identically, and like Spring you inject the interface, not the implementation.

TRAP — the captive dependency Injecting a **Scoped** service into a **Singleton** captures the first request's instance and keeps it forever. With a **DbContext** — which is not thread-safe — this produces intermittent, impossible-to-reproduce concurrency bugs in production. .NET will throw at startup if it detects this directly, but it cannot always see it.

Rule: **never inject a shorter-lived service into a longer-lived one.** `DbContext` is always Scoped.

Middleware order

The pipeline is sequential and **order is behaviour, not style:**



TRAP `UseAuthentication()` **must** come before `UseAuthorization()`. Authentication establishes identity; authorization checks it. Reversed, every `[Authorize]` endpoint returns 401 regardless of a valid token — and the error message tells you nothing useful. This is a genuinely common half-day debugging session.

4.4 CORS (L64 CORS Code, L65 CORS – 11 min)

Browsers enforce the **same-origin policy**: JavaScript on `http://localhost:3000` cannot read a response from `http://localhost:5000` unless that server explicitly permits it. Different port means different origin. Scheme, host and port must all match.

CORS (Cross-Origin Resource Sharing) is the server's way of granting that permission, via response headers.

This matters to you specifically: **your project is .NET + React**. The React dev server and the API run on different ports. Without CORS configured, every call from your front end fails with a console error that points at the browser rather than at the real cause.

TRAP A CORS failure looks like a network error in the browser console, but the request usually *reached the server and succeeded* — the browser simply refused to hand the response to your JavaScript. Check the Network tab: if you see a 200 with a response body but your code got an error, it is CORS. Postman never shows CORS errors because Postman is not a browser.

TRAP `.AllowAnyOrigin()` cannot be combined with `.AllowCredentials()` — the browser rejects it, by specification. If you need cookies or auth headers, you must name explicit origins, which is why the course defines separate Dev and Prod policies.

ON THE JOB Never ship `AllowAnyOrigin` to production. It is a standard penetration-test finding and will appear in a client security review. Name the origins. The course's two-policy Dev/Prod split is exactly the right pattern.

4.5 Two setup bugs the course calls out (L66, L67, L68)

L67 — InvariantGlobalization. The `webapi` template sometimes generates `<InvariantGlobalization>true</InvariantGlobalization>` in the `.csproj`. This strips ICU culture data and causes failures with culture-specific formatting or comparison. If you hit odd string or date behaviour, set it to `false`.

L66 — Mac/Linux users. SQL Server runs in Docker; the connection string uses SQL Authentication rather than `Trusted Connection`, since Windows Authentication does not exist there.

L68 — AutoMapper versioning. As covered in Module 3: registration syntax varies by major version. Check yours.

4.6 Your first controller (L62 First Custom Controller – 15 min)

```

using Microsoft.AspNetCore.Mvc;

namespace DotnetAPI.Controllers;

[ApiController]
[Route("[controller]")]
public class TestController : ControllerBase
{
    [HttpGet(Name = "TestConnection")]
    public string Connect() => "I am connected";
}

```

`ControllerBase` **not** `Controller`. `Controller` adds view-rendering support for MVC applications serving HTML. An API returns data, so `ControllerBase` is correct — it keeps the class free of machinery you will never use.

`[ApiController]` enables API-specific conventions: automatic 400 responses on model validation failure, inference of `[FromBody]` for complex parameters, and required attribute routing.

`[Route("[controller]")]` — the `[controller]` token is replaced with the class name minus the `Controller` suffix, so `TestController` serves `/Test`.

ON THE JOB The course uses `[Route("[controller]")]`, giving `/User`. Most production APIs use `[Route("api/[controller]")]` and lowercase URLs: `/api/user`. Follow your project's existing convention — but if you are starting something new, `api/` prefixing is the norm and makes routing/proxying far easier.

4.7 Models and the Users controller (L71 User Models — 17 min, L72 Users Controller — 18 min, L75 Namespaces)

```

namespace DotnetAPI.Models;

public partial class User
{
    public int    UserId    { get; set; }
    public string FirstName { get; set; } = string.Empty;
    public string LastName  { get; set; } = string.Empty;
    public string Email     { get; set; } = string.Empty;
    public string Gender    { get; set; } = string.Empty;
    public bool   Active    { get; set; }
}

```

`partial` allows a class to be split across files — useful when part is generated. Not essential here.

```

[ApiController]
[Route("[controller]")]
public class UserController : ControllerBase
{
    private readonly DataContextDapper _dapper;

    public UserController(IConfiguration config)
    {
        _dapper = new DataContextDapper(config);
    }

    [HttpGet("GetUsers")]
    public IEnumerable<User> GetUsers()
    {
        string sql = @"
            SELECT UserId, FirstName, LastName, Email, Gender, Active
            FROM TutorialAppSchema.Users";
        return _dapper.LoadData<User>(sql);
    }

    [HttpGet("GetSingleUser/{userId}")]
    public User GetSingleUser(int userId)
    {
        string sql = @"
            SELECT UserId, FirstName, LastName, Email, Gender, Active
            FROM TutorialAppSchema.Users
            WHERE UserId = " + userId.ToString();
        return _dapper.LoadDataSingle<User>(sql);
    }
}

```

TRAP — read this carefully The course concatenates `userId` into the SQL string at this stage, and it is *technically* safe here only because `userId` is typed `int` — a non-numeric value fails model binding before reaching this code.

Do not internalise this as a pattern. It is one refactor away from disaster: change the parameter to `string`, or copy the line for an email lookup, and you have a live SQL injection. Section 10 replaces all of this with `DynamicParameters`, which is the correct approach. Write it parameterised from the start:

```
string sql = "SELECT ... WHERE UserId = @UserId";
return _dapper.LoadDataSingle<User>(sql, new { UserId = userId });
```

Routing

```
[HttpGet("GetUsers")]           // GET /User/GetUsers
[HttpGet("GetSingleUser/{userId}")] // GET /User/GetSingleUser/5 - route parameter
[HttpGet]                       // GET /User
[HttpPost("AddUser")]            // POST /User/AddUser
```

Parameter binding sources:

Attribute	Source	Default for
[FromRoute]	URL path segment	Simple types matching a route token
[FromQuery]	Query string	Other simple types
[FromBody]	Request body JSON	Complex types (with [ApiController])
[FromHeader]	HTTP header	—

4.8 PUT and POST (L73 Put & Post — 36 min, the longest lecture in this section)

```
[HttpPut("EditUser")]
public IActionResult EditUser(User user)
{
    string sql = @"
        UPDATE TutorialAppSchema.Users
        SET FirstName = @FirstName,
            LastName  = @LastName,
            Email     = @Email,
            Gender    = @Gender,
            Active    = @Active
        WHERE UserId = @UserId";

    if (_dapper.ExecuteSql(sql, user))
    {
        return Ok();
    }
    throw new Exception("Failed to update user");
}

[HttpPost("AddUser")]
public IActionResult AddUser(UserToAddDto user)
{
    string sql = @"
        INSERT INTO TutorialAppSchema.Users
        (FirstName, LastName, Email, Gender, Active)
        VALUES (@FirstName, @LastName, @Email, @Gender, @Active)";

    if (_dapper.ExecuteSql(sql, user))
    {
        return Ok();
    }
    throw new Exception("Failed to add user");
}
```

IActionResult and result helpers

`IActionResult` lets a single action return different HTTP responses:

```
return Ok(); // 200
return Ok(user); // 200 with body
return Created("/user/{id}", user); // 201
return NoContent(); // 204
return BadRequest("Invalid email"); // 400
return Unauthorized(); // 401
return Forbid(); // 403
return NotFound(); // 404
return Conflict(); // 409
return StatusCode(500, "Failed"); // any code
```

`ActionResult<T>` gives you both: the type is documented for Swagger, and you can still return `NotFound()`.

ON THE JOB — the exception-throwing pattern The course throws a raw `Exception` on failure, which surfaces as a 500 with a stack trace in Development. It keeps the teaching focused but it is **not** production-acceptable:

- A 500 tells the client “we broke”, when the truth is often “your request was invalid” (400) or “that does not exist” (404).
- In Development the stack trace goes to the client — information disclosure.
- Nothing is logged in a structured, searchable way.

Production approach: return the correct status code, and add a **global exception handling middleware** that logs the exception with a correlation ID and returns a sanitised `ProblemDetails` response. On a PwC engagement there will be a logging standard — Serilog to Application Insights is the common stack. Ask what it is in your first week; do not invent your own.

Model validation

```
public class UserToAddDto
{
    [Required]
    [StringLength(50)]
    public string FirstName { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Range(0, 120)]
    public int Age { get; set; }
}
```

With `[ApiController]`, a failed validation automatically returns 400 with a structured error body before your action runs. You do not need to check `ModelState.IsValid` manually.

4.9 DTOs (L74 DTOs — 4 min, short but architecturally important)

A **Data Transfer Object** is a shape designed for a specific API operation, distinct from your database model.

```
public class UserToAddDto
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Gender { get; set; } = string.Empty;
    public bool Active { get; set; }
    // Deliberately no UserId — the database generates it
}
```

Why they matter:

1. **Security.** The model has fields the client must never set (`UserId`) or see (`PasswordHash`). Binding directly to your entity enables **over-posting**: a client adds `"isAdmin": true` to the JSON and the model binder sets it.
2. **Decoupling.** Your database schema can change without breaking the API contract.
3. **Shape.** The client may want `FullName` where the database stores two columns.
4. **Documentation.** Swagger shows precisely what each endpoint accepts.

ON THE JOB “Never bind directly to your entity model” is a reviewable rule at most firms and a standard security-review finding. The over-posting vulnerability is real and has caused well-publicised breaches. Always accept a DTO.

4.10 Delete (L76 User Delete — 10 min)

```
[HttpDelete("DeleteUser/{userId}")]
public IActionResult DeleteUser(int userId)
{
    string sql = "DELETE FROM TutorialAppSchema.Users WHERE UserId = @UserId";
    if (!_dapper.ExecuteSql(sql, new { UserId = userId }))
    {
        return Ok();
    }
    throw new Exception("Failed to delete user");
}
```

ON THE JOB — hard delete vs soft delete This physically removes the row. Most enterprise systems **soft delete** instead: `UPDATE ... SET Active = 0` or set a `DeletedAt` timestamp. Reasons: audit trails, regulatory retention, “undo”, and foreign keys that would otherwise break.

Note that the course’s `Users` table already has an `Active` column — that is the soft-delete flag. Ask early which your project uses; mixing the two produces orphaned data. For a firm doing audit and regulated work, soft delete is very often mandatory, and GDPR-style erasure requests are then handled by a separate, deliberate process.

4.11 The EF Core path (L77 EF Setup — 20 min, L78 EF User Controller — 19 min, L79 AutoMapper — 3 min)

The course now rebuilds the same controller using EF Core, so you can compare directly.

```
[ApiController]
[Route("[controller]")]
public class UserEFController : ControllerBase
{
    private readonly IUserRepository _userRepository;
    private readonly IMapper _mapper;

    public UserEFController(IUserRepository userRepository, IMapper mapper)
    {
        _userRepository = userRepository;
        _mapper = mapper;
    }

    [HttpGet("GetUsers")]
    public IEnumerable<User> GetUsers() => _userRepository.GetUsers();

    [HttpGet("GetSingleUser/{userId}")]
    public User GetSingleUser(int userId) => _userRepository.GetSingleUser(userId);

    [HttpPut("EditUser")]
    public IActionResult EditUser(User user)
    {
        User? userDb = _userRepository.GetSingleUser(user.UserId);
        if (userDb != null)
        {
            _mapper.Map(user, userDb); // copy onto the TRACKED entity
            if (_userRepository.SaveChanges())
            {
                return Ok();
            }
        }
        throw new Exception("Failed to update user");
    }

    [HttpPost("AddUser")]
    public IActionResult AddUser(UserToAddDto user)
    {
        User userDb = _mapper.Map<User>(user);
        _userRepository.AddEntity<User>(userDb);
        if (_userRepository.SaveChanges())
        {
            return Ok();
        }
        throw new Exception("Failed to add user");
    }

    [HttpDelete("DeleteUser/{userId}")]
    public IActionResult DeleteUser(int userId)
    {
        User? userDb = _userRepository.GetSingleUser(userId);
        if (userDb != null)
        {
            _userRepository.RemoveEntity<User>(userDb);
            if (_userRepository.SaveChanges())
            {
                return Ok();
            }
        }
        throw new Exception("Failed to delete user");
    }
}
```

Notice the shape of the EF update: **load the tracked entity, map onto it, save**. Mapping onto the tracked instance is what lets change tracking generate the `UPDATE`. Constructing a fresh `User` and calling `Update()` on it works too but overwrites every column, including ones the client never sent.

Compare the two controllers honestly:

- **Dapper** — you see exactly what SQL runs; more code; no change tracking; one round trip for an update.
- **EF Core** — less code; the update costs a read plus a write; the SQL is generated and you must trust it.

Neither is correct in the abstract. Being able to explain the trade-off in an interview or a design discussion is the point of the course teaching both.

4.12 The Beginner Assignment (L80 Assignment, L81 Solution — Dapper, L82 Solution — Entity Framework)

The assignment: build full CRUD for a second entity, both ways.

Do it before reading either solution. This is the single highest-value exercise in the first half of the course, because it forces you to build the whole vertical slice — model, DTO, repository, controller, SQL — unaided. Everything in Sections 8 and 10

assumes you can do this fluently.

Practice — Module 4

Replaces the Beginner Assignment (L80). Build these in your `DotnetAPI` project.

1. **Scaffold.** Create the web API project, delete the WeatherForecast sample, and get `dotnet watch run` serving Swagger.
2. **launchSettings.** Change the HTTP port to 5050. Confirm the change and then write one sentence on why this file has no effect in Azure.
3. **First controller.** Build `TestController` with a `GET` returning a string. Hit it in Swagger, then in Postman.
4. **Routing.** Add four actions to `TestController`: one with no route parameter, one with a route parameter, one with a query-string parameter, one taking a complex object from the body. Confirm which binding source each uses by default.
5. **CORS.** Configure Dev and Prod policies. Build a one-page HTML file with a `fetch()` call to your API, serve it from a different port, and observe the failure before the policy is applied and success afterwards. Then look at the Network tab and confirm the request actually succeeded.
6. **DI lifetimes.** Register the same service as Transient, Scoped and Singleton in turn, injecting a `Guid` generated in its constructor, and log it on each request. Explain the difference you observe.
7. **DI trap.** Deliberately inject a Scoped service into a Singleton. Record what happens at startup and explain why it is dangerous.
8. **Middleware order.** Swap `UseAuthentication()` and `UseAuthorization()`. Note what breaks and how unhelpful the error is.
9. **Dapper CRUD.** Build the full `UserController` with GET all, GET by ID, POST, PUT, DELETE. **Parameterise every query from the start** — do not follow the course's concatenation.
10. **DTOs.** Create `UserToAddDto` without `UserId`. Then try to POST a JSON body containing `"userId": 999` to an endpoint that binds directly to `User`, and observe over-posting in action. Write two sentences on why DTOs prevent it.
11. **Status codes.** Rewrite every action to return the correct code: 200 with body on GET, 201 on create, 204 on delete, 404 when the ID does not exist, 400 on validation failure. Remove every `throw new Exception`.
12. **Validation.** Add `[Required]`, `[EmailAddress]` and `[StringLength]` to your DTO. POST invalid data and examine the automatic 400 response body.
13. **EF path.** Build `UserEFController` with the same five operations using EF Core. Use the load-map-save pattern for updates.
14. **Compare.** Run both controllers' update operations and count the database round trips each makes. Write a short paragraph on when you would choose each.
15. **The assignment.** Build complete CRUD for a second entity — `Posts` — twice, once with Dapper and once with EF Core, without looking at lectures 81 or 82. Include DTOs, correct status codes and parameterised SQL.
16. **Soft delete.** Convert your delete to a soft delete setting `Active = 0`, and adjust `GetUsers` accordingly. Note what else in the application now needs to change.

Self-check — Module 4

1. What does idempotent mean, and which verbs are?
2. When do you return 401 and when 403?
3. Which status code for a successful POST, and which for a successful DELETE?
4. Why `ControllerBase` and not `Controller`?
5. Name three things `[ApiController]` does for you.
6. What does the `[controller]` token in a route resolve to?
7. Where did `Startup.cs` go, and what replaced it?
8. Name the three DI lifetimes and the correct one for a repository.
9. What is a captive dependency and why is it dangerous with `DbContext`?
10. Why must `UseAuthentication` precede `UseAuthorization`, and what is the symptom if reversed?
11. What is the same-origin policy, and why does it affect a .NET + React project specifically?
12. Why does a CORS error not appear in Postman?
13. Why can `AllowAnyOrigin` not be combined with `AllowCredentials`?
14. What is `launchSettings.json` for, and where does it not apply?
15. What is over-posting, and how do DTOs prevent it?
16. Give three reasons to use a DTO rather than your entity model.
17. What is wrong with `throw new Exception("Failed")` in a controller action?
18. What does `[FromBody]` bind by default under `[ApiController]`?
19. In the EF update pattern, why load the entity before mapping onto it?

20. How many database round trips does an EF update take versus a Dapper update?
21. What is soft delete, and why do regulated clients usually require it?
22. Which column in the course's `Users` table is already a soft-delete flag?

Module 5 — Repository Pattern, Authentication and JWT

Course section 8. Lectures 83–98. 4h 11m. The most important module in the course for employability: authentication done correctly is the thing every reviewer, auditor and client security team looks at first.

5.1 The repository pattern (L83 API Intermediate Theory, L84 UserRepository — 13 min, L85 IUserRepository — 10 min, L86 Repository Peripherals — 15 min)

The controllers in Module 4 had SQL inside them. That is fine for a tutorial and wrong for anything that ships:

- Controllers cannot be unit tested without a database.
- The same query gets duplicated across actions.
- Swapping Dapper for EF means rewriting every controller.
- Business logic and HTTP concerns are tangled together.

The **repository pattern** puts a named interface between the controller and the data access.

```
namespace DotnetAPI.Data;

public interface IUserRepository
{
    bool SaveChanges();
    void AddEntity<T>(T entityToAdd);
    void RemoveEntity<T>(T entityToRemove);
    IEnumerable<User> GetUsers();
    User GetSingleUser(int userId);
    UserSalary GetSingleUserSalary(int userId);
    UserJobInfo GetSingleUserJobInfo(int userId);
}
```

```
public class UserRepository : IUserRepository
{
    private readonly DbContextEF _entityFramework;

    public UserRepository(IConfiguration config)
    {
        _entityFramework = new DbContextEF(config);
    }

    public bool SaveChanges()
    {
        return _entityFramework.SaveChanges() > 0;
    }

    public void AddEntity<T>(T entityToAdd)
    {
        if (entityToAdd != null)
        {
            _entityFramework.Add(entityToAdd);
        }
    }

    public void RemoveEntity<T>(T entityToRemove)
    {
        if (entityToRemove != null)
        {
            _entityFramework.Remove(entityToRemove);
        }
    }

    public IEnumerable<User> GetUsers()
    {
        return _entityFramework.Users.ToList<User>();
    }

    public User GetSingleUser(int userId)
    {
        User? user = _entityFramework.Users
            .Where(u => u.UserId == userId)
            .FirstOrDefault<User>();

        if (user != null)
        {
            return user;
        }
        throw new Exception("Failed to Get User");
    }
}
```

Registered in `Program.cs`:

```
builder.Services.AddScoped<IUserRepository, UserRepository>();
```

The controller now depends on the interface only:

```
public UserEFController(IUserRepository userRepository, IMapper mapper)
{
    _userRepository = userRepository;
    _mapper = mapper;
}
```

JAVA BRIDGE This is exactly Spring Data's `UserRepository` interface plus `@Repository` implementation, injected into a `@RestController`. Identical pattern, identical reasoning. `AddScoped<IUserRepository, UserRepository>()` is what `@Repository` plus component scanning does implicitly.

ON THE JOB The generic `AddEntity<T>` / `RemoveEntity<T>` approach the course uses is pragmatic but loose — it leaks EF concepts into the interface and makes the repository a thin wrapper over `DbContext`. A more structured codebase either uses a generic `IRepository<T>` with typed operations, or drops the repository entirely for EF Core (since `DbSet<T>` is already a repository and `DbContext` is already a unit of work).

Have the argument ready: “Do we need a repository over EF Core?” is a genuine and recurring architecture debate. The strongest case for keeping one is testability and the ability to swap data access. The strongest case against is that it duplicates what EF already provides and tends to leak `IQueryable` anyway. Follow the project's existing structure; do not import a different pattern into an established codebase.

5.2 The Auth table and settings (L87 Auth Table SQL — 6 min, L88 Auth Settings and Models — 9 min)

```
CREATE TABLE TutorialAppSchema.Auth (
    Email NVARCHAR(50) NOT NULL PRIMARY KEY,
    PasswordHash VARBINARY(MAX) NOT NULL,
    PasswordSalt VARBINARY(MAX) NOT NULL
);
```

Three points of design worth stating explicitly:

1. **Separate table.** A profile query cannot leak credentials, because they are not in that table.
2. **VARBINARY, not NVARCHAR.** The hash and salt are raw bytes, not text. Storing them as text requires encoding and invites conversion bugs.
3. **No password column.** There is nowhere for a plaintext password to live, by construction.

The models:

```
public class UserForLoginDto
{
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

public class UserForRegistrationDto : UserForLoginDto
{
    public string PasswordConfirm { get; set; } = string.Empty;
}

public class UserForLoginConfirmationDto
{
    public byte[] PasswordHash { get; set; } = new byte[0];
    public byte[] PasswordSalt { get; set; } = new byte[0];
}
```

Configuration:

```
{
  "AppSettings": {
    "PasswordKey": "a long random string, never committed",
    "TokenKey": "a different long random string, never committed"
  }
}
```

TRAP — the one that becomes a security incident Both keys must be **long, random, and never committed to source control**. Use User Secrets locally and Azure Key Vault or App Service configuration in deployment (Modules 3 and 8).

A `TokenKey` in a Git repository means anyone who can read the repository can mint valid tokens for any user. At a Big Four firm this is a reportable security incident, and a client's repository scanning will find it. If it ever happens by accident: rotate the key immediately and tell your team lead. Do not quietly force-push and hope.

5.3 Password management (L89 Password Management — 38 min, the longest)

lecture in this section and the most important)

Why you never store a plaintext password

If the database is compromised, every password is exposed — and because people reuse passwords, you have also compromised those users' email, banking and everything else. This is why breaches are catastrophic beyond the breached system.

Hashing, not encryption

Encryption is reversible with a key. **Hashing is one-way** — you cannot recover the input from the output. Passwords must be hashed, never encrypted, because your system never needs to know the password. It only needs to know whether a supplied password produces the same hash.

Why salt

Without a salt, identical passwords produce identical hashes. An attacker who cracks one has cracked every account using that password, and precomputed **rainbow tables** map common hashes straight back to their inputs.

A **salt** is unique random bytes generated per user and stored alongside the hash. It makes identical passwords hash differently and renders precomputed tables useless — each password must be attacked individually.

The course's implementation

```
using System.Security.Cryptography;

// Generate a unique salt per user
byte[] passwordSalt = new byte[128 / 8];
using (RandomNumberGenerator rng = RandomNumberGenerator.Create())
{
    rng.GetNonZeroBytes(passwordSalt);
}

// Derive the hash
private byte[] GetPasswordHash(string password, byte[] passwordSalt)
{
    string passwordSaltPlusString = _config.GetSection("AppSettings:PasswordKey").Value
        + Convert.ToBase64String(passwordSalt);

    return KeyDerivation.Pbkdf2(
        password: password,
        salt: Encoding.ASCII.GetBytes(passwordSaltPlusString),
        prf: KeyDerivationPrf.HMACSHA256,
        iterationCount: 1000000,
        numBytesRequested: 256 / 8
    );
}
```

The pieces:

- **RandomNumberGenerator** — cryptographically secure randomness. **Never** `System.Random` for anything security-related; it is predictable from its seed.
- **PBKDF2** — Password-Based Key Derivation Function 2. A deliberately slow hash.
- **iterationCount: 1000000** — one million iterations. Slowness is the point: it costs you milliseconds per login and costs an attacker with a stolen database an enormous amount per guess.
- **The PasswordKey from config** — an application-level secret mixed in alongside the per-user salt. Sometimes called a *pepper*. It means a stolen database alone is not enough; the attacker also needs the application's configuration.

Verifying at login

```
byte[] passwordHash = GetPasswordHash(userForLogin.Password, userForLoginConfirmation.PasswordSalt);

for (int index = 0; index < passwordHash.Length; index++)
{
    if (passwordHash[index] != userForLoginConfirmation.PasswordHash[index])
    {
        return StatusCode(401, "Incorrect password!");
    }
}
```

Re-derive the hash from the supplied password using the **stored salt**, then compare byte by byte.

ON THE JOB — two improvements worth knowing

1. Use a fixed-time comparison. The loop above exits early on the first differing byte, so its runtime leaks information about how many leading bytes matched. This is a **timing attack**, and while it is difficult to exploit over a network, it is trivial to avoid:

```
if (CryptographicOperations.FixedTimeEquals(computedHash, storedHash)) { ... }
```

Use `CryptographicOperations.FixedTimeEquals`. It is one line, it is in the BCL, and a security reviewer will look for it.

2. Consider ASP.NET Core Identity instead. For a real project, Microsoft's Identity framework gives you password hashing, lockout after failed attempts, password reset tokens, two-factor authentication, external providers and a maintained upgrade path for hashing algorithms. The course implements it by hand so you understand what is happening underneath — which is genuinely valuable — but hand-rolled authentication in a client deliverable will be questioned. **Know how it works; use the framework.**

Related: the modern recommendation for new systems is **Argon2id** or **bcrypt** rather than PBKDF2, though PBKDF2 with a high iteration count remains acceptable and is FIPS-approved, which matters for some government and financial clients.

TRAP — never log or return a password Not in a log statement, not in an exception message, not in a DTO that gets serialised. Once a password is in an application log it is in your log aggregation system, your backups, and possibly a third-party SaaS. Scrub it at the boundary. This is a standard finding in security reviews.

Registration (L91 Registration With Details — 10 min, L120 Auth Register in Section 10)

```
[AllowAnonymous]
[HttpPost("Register")]
public IActionResult Register(UserForRegistrationDto userForRegistration)
{
    if (userForRegistration.Password != userForRegistration.PasswordConfirm)
    {
        return StatusCode(400, "Passwords do not match!");
    }

    // Does the email already exist?
    string sqlCheckUserExists = @"
        SELECT Email FROM TutorialAppSchema.Auth WHERE Email = @Email";
    IEnumerable<string> existingUsers =
        _authHelper.LoadData<string>(sqlCheckUserExists, new { Email = userForRegistration.Email });

    if (existingUsers.Count() > 0)
    {
        return StatusCode(409, "User with this email already exists!");
    }

    // ... generate salt, derive hash, insert into Auth, then insert into Users
}
```

ON THE JOB — user enumeration Returning “user already exists” on registration tells an attacker which email addresses have accounts. For a consumer application that is a real privacy concern and security reviewers flag it; the mitigation is to return a generic success message and send an email either way.

For an internal enterprise tool it is usually accepted, because the user list is not secret and the usability cost of hiding it is high. Know the trade-off and ask which applies. The same reasoning applies to login: **“Invalid email or password”** as a single message, never “no such user” versus “wrong password”.

5.4 JWT — creation (L93 JWT Token Creation — 23 min)

What a JWT is

A **JSON Web Token** is a self-contained, cryptographically signed credential. Because REST is stateless, the server keeps no session — the client sends the token on every request and the server verifies the signature.

Three base64url-encoded parts separated by dots: `header.payload.signature`.

```
eyJhbGciOiJIUzUxMiJ9.eyJ1c2VySWQiOiIxIn0.Xk9s...
```

- **Header** — algorithm and token type.
- **Payload** — the **claims**: `userId`, issued-at, expiry, roles.
- **Signature** — HMAC of header and payload using the server's secret key.

TRAP — the misunderstanding that causes breaches A JWT payload is encoded, not encrypted. Anyone holding the token can decode and read every claim — paste one into `jwt.io` and see. The signature prevents *tampering*, not *reading*.

Never put anything sensitive in a JWT payload. No passwords, no personal data, no internal identifiers you would not show the user. A user ID and a role is the right level.

Creating it

```

private string CreateToken(int userId)
{
    Claim[] claims = new Claim[]
    {
        new Claim("userId", userId.ToString())
    };

    string? tokenKeyString = _config.GetSection("AppSettings:TokenKey").Value;

    SymmetricSecurityKey tokenKey = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(tokenKeyString ?? ""));

    SigningCredentials credentials = new SigningCredentials(
        tokenKey,
        SecurityAlgorithms.HmacSha512Signature
    );

    SecurityTokenDescriptor descriptor = new SecurityTokenDescriptor()
    {
        Subject = new ClaimsIdentity(claims),
        SigningCredentials = credentials,
        Expires = DateTime.Now.AddDays(1)
    };

    JwtSecurityTokenHandler tokenHandler = new JwtSecurityTokenHandler();
    SecurityToken token = tokenHandler.CreateToken(descriptor);

    return tokenHandler.WriteToken(token);
}

```

ON THE JOB Token lifetime. One day is long for a production access token. The standard pattern is a **short-lived access token** (15 minutes to an hour) plus a **long-lived refresh token** that can be revoked server-side. This limits the damage window if a token is stolen — which matters because a signed JWT cannot be invalidated before it expires. There is no “log out everyone” without a revocation list or a key rotation.

Use `DateTime.UtcNow.AddHours(1)`, not `DateTime.Now`. `Now` is server-local. Deploy to an Azure region in a different timezone and your tokens expire at the wrong moment — or are already expired on issue. Module 1 flagged this; this is where it bites.

Symmetric vs asymmetric. HMAC-SHA512 with a shared secret is fine when one service both issues and validates. If several services must validate tokens they do not issue, use asymmetric signing (RSA/ECDSA) so only the issuer holds the private key. This is what identity providers like Entra ID do.

5.5 JWT — validation (*L94 JWT Token Validation — 34 min*)

```

// Program.cs
string? tokenKeyString = builder.Configuration.GetSection("AppSettings:TokenKey").Value;

TokenValidationParameters tokenValidationParameters = new TokenValidationParameters()
{
    IssuerSigningKey = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(tokenKeyString ?? "")),
    ValidateIssuer = false,
    ValidateIssuerSigningKey = true,
    ValidateAudience = false
};

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = tokenValidationParameters;
    });

```

Then in the pipeline, **in this order**:

```

app.UseAuthentication();
app.UseAuthorization();

```

Protecting endpoints:

```

[Authorize] // controller-wide
public class UserController : ControllerBase
{
    [AllowAnonymous] // explicit exception
    [HttpPost("Login")]
    public IActionResult Login(UserForLoginDto dto) { }
}

```

Reading claims inside an action:

```

string? userId = User.FindFirst("userId")?.Value;

```

ON THE JOB `ValidateIssuer = false` and `ValidateAudience = false` are **development shortcuts**. In production, set an issuer and audience and validate both — otherwise a token signed with the same key by any other system is accepted by yours. Also set `ClockSkew` deliberately; the default five-minute tolerance is often more than you want.

Many enterprise projects do not hand-roll any of this: they federate to **Microsoft Entra ID** (formerly Azure AD) and validate tokens issued by it. Given PwC's Microsoft estate, this is a very likely shape for your project. Understanding the mechanics from this course means you will understand what Entra is doing rather than treating it as magic.

ON THE JOB — where the React side stores the token Since your project is .NET + React, this will come up. `localStorage` is convenient and vulnerable to XSS — any injected script can read it. An **httpOnly cookie** is not readable by JavaScript but requires CSRF protection and correct `SameSite` settings. There is no free option; the mature answer is `httpOnly`, `Secure`, `SameSite=Strict` cookies with CSRF tokens, or short-lived tokens in memory with a refresh flow. Ask what the project has decided rather than choosing unilaterally.

5.6 Login (L90 Login — 14 min)

```
[AllowAnonymous]
[HttpPost("Login")]
public IActionResult Login(UserForLoginDto userForLogin)
{
    string sqlForHashAndSalt = @"
        SELECT PasswordHash, PasswordSalt
        FROM TutorialAppSchema.Auth
        WHERE Email = @Email";

    UserForLoginConfirmationDto userForConfirmation = _dapper
        .LoadDataSingle<UserForLoginConfirmationDto>(
            sqlForHashAndSalt, new { Email = userForLogin.Email });

    byte[] passwordHash = _authHelper.GetPasswordHash(
        userForLogin.Password, userForConfirmation.PasswordSalt);

    if (!CryptographicOperations.FixedTimeEquals(passwordHash, userForConfirmation.PasswordHash))
    {
        return StatusCode(401, "Incorrect password!");
    }

    string userIdSql = "SELECT UserId FROM TutorialAppSchema.Users WHERE Email = @Email";
    int userId = _dapper.LoadDataSingle<int>(userIdSql, new { Email = userForLogin.Email });

    return Ok(new Dictionary<string, string>
    {
        { "token", _authHelper.CreateToken(userId) }
    });
}
```

ON THE JOB — three production additions

Rate limiting. Without it, an attacker can attempt unlimited passwords. .NET 7+ has built-in rate limiting middleware; use it on login specifically.

Account lockout. Lock or delay after N consecutive failures. ASP.NET Core Identity provides this.

Audit logging. Log the outcome of every authentication attempt — user, timestamp, source IP, success or failure, **never the password**. For a firm doing audit work, this is usually a hard requirement rather than a nice-to-have.

5.7 Helpers (L95 Helpers — 7 min)

The course extracts `GetPasswordHash` and `CreateToken` into an `AuthHelper` class registered with DI, so both the auth controller and the registration flow share one implementation.

The principle: **one implementation of a security-critical operation**. Duplicated hashing code is how one copy gets fixed and the other does not.

5.8 Posts (L96 Posts Model — 14 min, L97 Posts Controller — 44 min, the longest lecture in the course, L98 Posts Search — 10 min)

```
public class Post
{
    public int PostId { get; set; }
    public int UserId { get; set; }
    public string PostTitle { get; set; } = string.Empty;
    public string PostContent { get; set; } = string.Empty;
    public DateTime PostCreated { get; set; }
    public DateTime PostUpdated { get; set; }
}
```

The Posts controller is long because it combines everything so far: CRUD, both data-access paths, authorization, and

ownership checks.

The authorization idea that matters most:

```
[HttpPut("EditPost")]
public IActionResult EditPost(PostToEditDto postToEdit)
{
    string sql = @"
        UPDATE TutorialAppSchema.Posts
        SET PostTitle = @PostTitle,
            PostContent = @PostContent,
            PostUpdated = GETDATE()
        WHERE PostId = @PostId
            AND UserId = @UserId";           // ownership enforced in the WHERE clause

    // UserId comes from the TOKEN, never from the request body
    string? userId = User.FindFirst("userId")?.Value;
    // ...
}
```

TRAP — insecure direct object reference (IDOR) Take the user's identity from the token, never from the request body.

If the client sends `userId` and you trust it, any authenticated user can edit any other user's posts by changing a number in the JSON. This is one of the most common real-world API vulnerabilities and it appears in the OWASP API Security Top 10 as "Broken Object Level Authorization".

The pattern above is correct: `userId` comes from `User.FindFirst("userId")`, and the `WHERE` clause enforces ownership at the database level so a mismatched ID simply updates zero rows.

Search (L98)

```
[HttpGet("Posts/{userId}/{searchParam}")]
public IEnumerable<Post> GetPosts(int userId = 0, string searchParam = "None")
{
    string sql = @"
        SELECT PostId, UserId, PostTitle, PostContent, PostCreated, PostUpdated
        FROM TutorialAppSchema.Posts WHERE 1=1";

    if (userId != 0)
    {
        sql += " AND UserId = @UserId";
    }
    if (searchParam.ToLower() != "none")
    {
        sql += " AND (PostTitle LIKE @Search OR PostContent LIKE @Search)";
    }
    // parameters supplied via DynamicParameters
}
```

The `WHERE 1=1` trick makes conditional `AND` clauses easy to append without tracking whether you have written `WHERE` yet. It costs nothing — the optimiser removes it.

TRAP `LIKE '%term%'` with a leading wildcard **cannot use an index** and scans the whole table. Acceptable on a small table; a serious problem at scale. The production answer is SQL Server **Full-Text Search** (`CONTAINS`, `FREETEXT`) or a dedicated search index. Given you have used Elasticsearch, you already know the shape of that answer.

ON THE JOB This endpoint returns every matching row with no pagination. On a real dataset that is a memory and bandwidth problem and a denial-of-service vector. **Always paginate list endpoints** — `OFFSET/FETCH NEXT` in SQL, `page` and `pageSize` query parameters in the API, and a total count in the response. Section 10 improves the dynamic query building, but pagination is something you should add yourself.

Practice — Module 5

1. **Repository.** Refactor your `UserController` so all data access sits behind `IUserRepository`. Register it Scoped. Confirm the controller has no SQL and no `DbContext` reference.
2. **Testability.** Write a unit test for a controller action using a hand-written fake `IUserRepository`. Note that this was impossible before the refactor.
3. **Auth table.** Create the `Auth` table. Write one sentence each on why it is separate from `Users`, why the columns are `VARBINARY`, and why there is no `Password` column.
4. **Salt.** Generate a cryptographic salt with `RandomNumberGenerator`. Then generate one with `System.Random` and write a comment explaining why the second is unacceptable.
5. **Hash.** Implement `GetPasswordHash` with PBKDF2, HMACSHA256, a high iteration count, and the `PasswordKey` from configuration. Hash the same password twice with different salts and confirm the outputs differ.
6. **Registration.** Build the register endpoint: confirm passwords match, reject duplicate emails, generate salt, derive hash, insert into `Auth`, insert into `Users`. Return the correct status codes.

7. **Login.** Build the login endpoint. Use `CryptographicOperations.FixedTimeEquals` rather than the course's byte loop, and write a comment explaining the timing attack.
8. **Error messages.** Make login return an identical message for "no such user" and "wrong password". Explain in a comment what that prevents.
9. **Token creation.** Implement `CreateToken` with a `userId` claim. Decode the resulting token at jwt.io and confirm you can read the payload. Write one sentence on what that proves about what may go in a JWT.
10. **Token expiry.** Set expiry with `DateTime.UtcNow.AddHours(1)`. Explain in a comment what breaks if you use `DateTime.Now` on a server in a different timezone.
11. **Validation.** Wire up JWT bearer authentication in `Program.cs`. Add `[Authorize]` to a controller and confirm an unauthenticated request returns 401 and an authenticated one succeeds.
12. **Middleware order.** Swap `UseAuthentication` and `UseAuthorization`, observe the failure with a valid token, and restore the order.
13. **Tampering.** Take a valid token, change one character in the payload, and send it. Confirm the request is rejected and explain which of the three parts caught it.
14. **Claims.** Read `userId` from the token inside an action using `User.FindFirst`.
15. **IDOR.** Build an "edit post" endpoint that takes `userId` from the request body. Log in as user A and edit user B's post. Then fix it by taking `userId` from the token and enforcing ownership in the `WHERE` clause. Confirm the attack now fails.
16. **Posts CRUD.** Build the full Posts controller: get all, get by user, get by ID, search, create, edit, delete — with ownership enforced on every write.
17. **Pagination.** Add `page` and `pageSize` parameters to your posts list endpoint using `OFFSET / FETCH NEXT`, and return a total count alongside the results.
18. **Search performance.** Run your `LIKE '%term%'` search against a table with a large number of rows and look at the execution plan. Identify the scan and write one sentence on what you would use instead at scale.
19. **Secrets.** Move `TokenKey` and `PasswordKey` into User Secrets. Confirm nothing sensitive remains in `appsettings.json`, then grep your whole repository to be certain.

Self-check — Module 5

1. Give three reasons to put a repository between the controller and the database.
2. What is the argument *against* a repository on top of EF Core?
3. Why is `Auth` a separate table, and why are its columns `VARBINARY`?
4. What is the difference between hashing and encryption, and which do passwords need?
5. What is a salt and what specific attack does it defeat?
6. What is a pepper, and where does the course store it?
7. Why is a million PBKDF2 iterations a feature rather than a performance bug?
8. Why must you never use `System.Random` for a salt?
9. What is a timing attack, and what one-line change avoids it?
10. Name three things ASP.NET Core Identity would give you that the course's hand-rolled auth does not.
11. What are the three parts of a JWT and what does each contain?
12. Is a JWT payload encrypted? What follows from the answer?
13. What does the signature actually protect against?
14. Why is a one-day token lifetime risky, and what is the standard alternative?
15. Why can a JWT not be revoked before expiry, and what do systems do about it?
16. When would you use asymmetric rather than symmetric token signing?
17. Why are `ValidateIssuer = false` and `ValidateAudience = false` unacceptable in production?
18. What does `[AllowAnonymous]` do on an action inside an `[Authorize]` controller?
19. What is user enumeration, and when is it an acceptable risk?
20. What is IDOR, and what single rule prevents it?
21. Why must `userId` come from the token rather than the request body?
22. Why can `LIKE '%term%'` not use an index?
23. Why must list endpoints be paginated?
24. What are the trade-offs between `localStorage` and httpOnly cookies for storing a token in React?

Module 6 — Stored Procedures

Course section 9. Lectures 99–110. 1h 50m. Short, dense, and the foundation for everything in Section 10.

6.1 What a stored procedure is, and why (L99 Stored Procedure Creation — 11 min)

A **stored procedure** is named, parameterised T-SQL saved in the database and executed by name.

```
CREATE PROCEDURE TutorialAppSchema.spUsers_Get
AS
BEGIN
    SELECT UserId, FirstName, LastName, Email, Gender, Active
    FROM TutorialAppSchema.Users;
END
GO
```

```
EXEC TutorialAppSchema.spUsers_Get;
```

Management:

```
CREATE PROCEDURE ...      -- fails if it already exists
ALTER PROCEDURE ...      -- modify an existing one
DROP PROCEDURE ...       -- delete

-- The idempotent form for deploy scripts
CREATE OR ALTER PROCEDURE TutorialAppSchema.spUsers_Get
AS
BEGIN
    ...
END
GO
```

CREATE OR ALTER is what you want in a script that runs repeatedly. Without it, a redeploy fails on the second run.

Naming. The course uses `spTableName_Action`: `spUsers_Get`, `spUsers_Upsert`, `spUsers_Delete`, `spPosts_Get`. Consistent naming means you can find every procedure touching a table alphabetically, which matters once there are two hundred of them.

TRAP Do not prefix with `sp_`. That prefix is reserved for Microsoft’s own system procedures in the `master` database, and SQL Server looks there *first* when resolving a name beginning with `sp_` — costing a lookup on every call, and risking a name collision with a future system procedure. `sp` without the underscore, as the course uses, is fine.

Why use them

Argument	Detail
Performance	The execution plan is compiled and cached. Repeated calls skip parsing and planning.
Security	Grant EXECUTE on the procedure without granting SELECT on the underlying tables. The application account never touches the tables directly.
SQL injection	Parameters are strongly typed and never concatenated into the statement.
Maintainability	Query changes deploy to the database without rebuilding and redeploying the application.
Network	One procedure name instead of a large SQL string on every call.
Encapsulation	Multi-statement logic and transactions live in one atomic, named unit.

The security argument is the strongest in an enterprise context. An application account with **EXECUTE** on procedures and nothing else cannot run an arbitrary `SELECT * FROM Auth` even if the application is compromised. This is why financial, government and healthcare clients so often mandate stored procedures.

The arguments against — know both sides

- **Logic in two places.** Business rules split between C# and T-SQL are harder to reason about and test.
- **Version control.** Procedures live in the database; keeping them in Git requires discipline and tooling.
- **Testing.** Unit testing T-SQL is harder than unit testing C#.
- **Portability.** T-SQL procedures tie you to SQL Server.
- **Debugging.** Poorer tooling than stepping through C#.

ON THE JOB “Stored procedures or ORM-generated SQL?” is one of the standing arguments in enterprise .NET, and you will meet strong opinions on both sides. The mature position: **stored procedures where security boundaries, complex multi-statement logic, or heavy reporting queries justify them; ORM for straightforward CRUD**. What matters far more than your preference is that you follow the existing convention in the codebase. A graduate who introduces a second pattern into an established codebase creates work for everyone.

Ask in week one: “Are procedures source-controlled, and how do schema changes get deployed?” The answer tells you a great deal about the project’s engineering maturity.

6.2 Parameters (L100 Parameters — 7 min)

```
CREATE PROCEDURE TutorialAppSchema.spUsers_Get
    @UserId INT
AS
BEGIN
    SELECT UserId, FirstName, LastName, Email, Gender, Active
    FROM TutorialAppSchema.Users
    WHERE UserId = @UserId;
END
GO
```

```
EXEC TutorialAppSchema.spUsers_Get @UserId = 1;
EXEC TutorialAppSchema.spUsers_Get 1;           -- positional, works but less readable
```

Always use named parameters at the call site. Positional arguments break silently when someone adds a parameter.

Output parameters:

```
CREATE PROCEDURE TutorialAppSchema.spUsers_Insert
    @FirstName NVARCHAR(50),
    @LastName NVARCHAR(50),
    @NewUserId INT OUTPUT
AS
BEGIN
    INSERT INTO TutorialAppSchema.Users (FirstName, LastName, Email, Gender, Active)
    VALUES (@FirstName, @LastName, '', '', 1);

    SET @NewUserId = SCOPE_IDENTITY();
END
GO
```

6.3 Nullable parameters (L101 Nullable Parameters — 5 min)

This is the technique the whole of Section 10 is built on. Give parameters a `NULL` default and let the `WHERE` clause treat `NULL` as “do not filter on this”:

```
CREATE PROCEDURE TutorialAppSchema.spUsers_Get
    @UserId INT = NULL,
    @Active BIT = NULL
AS
BEGIN
    SELECT UserId, FirstName, LastName, Email, Gender, Active
    FROM TutorialAppSchema.Users
    WHERE (@UserId IS NULL OR UserId = @UserId)
    AND (@Active IS NULL OR Active = @Active);
END
GO
```

One procedure now serves every combination:

```
EXEC TutorialAppSchema.spUsers_Get;           -- all users
EXEC TutorialAppSchema.spUsers_Get @UserId = 1; -- one user
EXEC TutorialAppSchema.spUsers_Get @Active = 1; -- active users
EXEC TutorialAppSchema.spUsers_Get @UserId = 1, @Active = 1; -- both filters
```

The pattern `WHERE (@Param IS NULL OR Column = @Param)` reads as: *if no value was supplied, this condition is always true and filters nothing; if a value was supplied, filter on it.*

TRAP — parameter sniffing, and the fix This pattern has a real performance cost at scale. SQL Server compiles **one** execution plan based on the parameter values from the *first* call and caches it. A plan optimised for `@UserId = 1` (an index seek returning one row) is terrible for `@UserId = NULL` (which should scan), and vice versa. Symptom: a query that is fast for weeks and then suddenly slow for no apparent reason, usually after a server restart or statistics update.

Fixes, in rough order of preference: - **OPTION (RECOMPILE)** on the statement — recompiles per execution. Costs a little CPU, gives a correct plan every time. Usually the right answer for this pattern. - **OPTIMIZE FOR UNKNOWN** — uses average statistics rather than the sniffed value. - Separate procedures for genuinely different query shapes.

```
WHERE (@UserId IS NULL OR UserId = @UserId)
OPTION (RECOMPILE);
```

Knowing this makes you noticeably more credible than someone who has only learned the pattern. It is a genuine production issue, not a textbook curiosity.

6.4 JOIN and OUTER APPLY (L102 — 11 min)

```
CREATE PROCEDURE TutorialAppSchema.spUsers_Get
    @UserId INT = NULL
AS
BEGIN
    SELECT u.UserId, u.FirstName, u.LastName, u.Email, u.Gender, u.Active,
           s.Salary,
           j.JobTitle, j.Department
    FROM TutorialAppSchema.Users AS u
         LEFT JOIN TutorialAppSchema.UserSalary AS s ON u.UserId = s.UserId
         LEFT JOIN TutorialAppSchema.UserJobInfo AS j ON u.UserId = j.UserId
    WHERE (@UserId IS NULL OR u.UserId = @UserId);
END
GO
```

OUTER APPLY is T-SQL’s correlated join. Unlike a normal join, the right-hand expression can reference columns from the left — which lets you do “top N per row”:

```
SELECT u.UserId, u.FirstName, p.PostTitle, p.PostCreated
FROM TutorialAppSchema.Users AS u
     OUTER APPLY (
         SELECT TOP 1 PostTitle, PostCreated
         FROM TutorialAppSchema.Posts
         WHERE UserId = u.UserId           -- references the OUTER query
         ORDER BY PostCreated DESC
     ) AS p;
```

CROSS APPLY behaves like **INNER JOIN** (drops rows with no match); **OUTER APPLY** behaves like **LEFT JOIN** (keeps them with NULLs).

Use **APPLY** for: most-recent-child-per-parent, top-N-per-group, and calling a table-valued function per row. A plain **JOIN** cannot express any of these.

ON THE JOB **OUTER APPLY** and window functions solve overlapping problems. **ROW_NUMBER() OVER (PARTITION BY ...)** is often cleaner for top-N-per-group; **APPLY** is better when the subquery is expensive and you genuinely want it evaluated per row, or when calling a table-valued function. Knowing both and when each is clearer is a solid intermediate SQL signal.

6.5 Temp tables (L103 Temp Table — 16 min)

```
-- Local temp table: visible to this session only, dropped when it ends
CREATE TABLE #TempUsers (
    UserId INT,
    FirstName NVARCHAR(50),
    Salary DECIMAL(18,4)
);

INSERT INTO #TempUsers (UserId, FirstName, Salary)
SELECT u.UserId, u.FirstName, s.Salary
FROM TutorialAppSchema.Users u
     JOIN TutorialAppSchema.UserSalary s ON u.UserId = s.UserId;

SELECT * FROM #TempUsers WHERE Salary > 50000;

DROP TABLE #TempUsers;
```

Form	Scope
<code>#TableName</code>	Local — current session only
<code>##TableName</code>	Global — all sessions. Rarely a good idea
<code>@TableVariable</code>	Table variable — scoped to the batch
<code>WITH cte AS (...)</code>	CTE — scoped to one statement

Choosing between them:

- **CTE** — readability, a single statement, recursion. No statistics, no indexes, re-evaluated on each reference.
- **Table variable** — small result sets (roughly under a thousand rows). Minimal logging, but poor cardinality estimates historically, which can produce bad plans.
- **Temp table** — larger result sets, when you need indexes, when you reference the result multiple times. Has statistics, so the optimiser plans properly.

Rule of thumb: if the intermediate set is large or you use it more than once, temp table. Otherwise CTE for clarity.

6.6 The upsert pattern (*L104 User Upsert — 17 min, L105 User Upsert Peripherals — 9 min*)

Upsert = update if it exists, insert if it does not. One endpoint and one procedure handle both create and edit, which is exactly what a single “save” button in a React form wants.

```
CREATE PROCEDURE TutorialAppSchema.spUser_Upsert
    @FirstName NVARCHAR(50),
    @LastName NVARCHAR(50),
    @Email NVARCHAR(50),
    @Gender NVARCHAR(50),
    @Active BIT,
    @JobTitle NVARCHAR(50),
    @Department NVARCHAR(50),
    @Salary DECIMAL(18,4),
    @UserId INT = NULL
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM TutorialAppSchema.Users WHERE UserId = @UserId)
    BEGIN
        INSERT INTO TutorialAppSchema.Users
            (FirstName, LastName, Email, Gender, Active)
        VALUES (@FirstName, @LastName, @Email, @Gender, @Active);

        SET @UserId = SCOPE_IDENTITY();

        INSERT INTO TutorialAppSchema.UserSalary (UserId, Salary)
        VALUES (@UserId, @Salary);

        INSERT INTO TutorialAppSchema.UserJobInfo (UserId, JobTitle, Department)
        VALUES (@UserId, @JobTitle, @Department);
    END
    ELSE
    BEGIN
        UPDATE TutorialAppSchema.Users
        SET FirstName = @FirstName,
            LastName = @LastName,
            Email = @Email,
            Gender = @Gender,
            Active = @Active
        WHERE UserId = @UserId;

        UPDATE TutorialAppSchema.UserSalary
        SET Salary = @Salary
        WHERE UserId = @UserId;

        UPDATE TutorialAppSchema.UserJobInfo
        SET JobTitle = @JobTitle, Department = @Department
        WHERE UserId = @UserId;
    END
END
GO
```

“Peripherals” in lecture 105 means the related tables — `UserSalary` and `UserJobInfo` — which must be written alongside the main record.

TRAP — this procedure is not atomic and has a race condition Two problems a reviewer will find:

1. No transaction. If the `Users` insert succeeds and the `UserSalary` insert fails, you have a user with no salary row and no way to know it happened. Every multi-statement write belongs in a transaction:

```
BEGIN TRANSACTION;
BEGIN TRY
    -- all the inserts/updates
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    THROW;
END CATCH
```

2. Check-then-act race. Between `IF NOT EXISTS` and the `INSERT`, another connection can insert the same row. Under concurrency this produces duplicates or primary-key violations. Fixes: `MERGE` with `HOLDLOCK` (though `MERGE` has its own documented bugs and many DBAs avoid it), or a `WITH (UPDLOCK, HOLDLOCK)` hint on the existence check, or a unique constraint plus catching the violation.

The course's version teaches the pattern clearly. Adding the transaction is a genuine, easily-defended improvement to make on a real project.

6.7 The remaining procedures (L106 User Delete, L107 Posts Get, L108 Post Upsert, L109 Post Delete, L110 Auth)

```
CREATE PROCEDURE TutorialAppSchema.spUser_Delete
    @UserId INT
AS
BEGIN
    DELETE FROM TutorialAppSchema.UserSalary WHERE UserId = @UserId;
    DELETE FROM TutorialAppSchema.UserJobInfo WHERE UserId = @UserId;
    DELETE FROM TutorialAppSchema.Users WHERE UserId = @UserId;
END
GO
```

Note the deletion order: **children before parents**, or foreign key constraints reject it. (This also belongs in a transaction.)

```
CREATE PROCEDURE TutorialAppSchema.spPosts_Get
    @UserId INT = NULL,
    @PostId INT = NULL,
    @SearchValue NVARCHAR(500) = NULL
AS
BEGIN
    SELECT PostId, UserId, PostTitle, PostContent, PostCreated, PostUpdated
    FROM TutorialAppSchema.Posts
    WHERE (@UserId IS NULL OR UserId = @UserId)
    AND (@PostId IS NULL OR PostId = @PostId)
    AND (@SearchValue IS NULL
        OR PostTitle LIKE '%' + @SearchValue + '%'
        OR PostContent LIKE '%' + @SearchValue + '%');
END
GO
```

One procedure serves: all posts, one user's posts, a single post, and search — the nullable-parameter pattern doing exactly what it was designed for.

```
CREATE PROCEDURE TutorialAppSchema.spPost_Upsert
    @UserId INT,
    @PostTitle NVARCHAR(255),
    @PostContent NVARCHAR(MAX),
    @PostId INT = NULL
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM TutorialAppSchema.Posts WHERE PostId = @PostId)
    BEGIN
        INSERT INTO TutorialAppSchema.Posts
            (UserId, PostTitle, PostContent, PostCreated, PostUpdated)
        VALUES (@UserId, @PostTitle, @PostContent, GETDATE(), GETDATE());
    END
    ELSE
    BEGIN
        UPDATE TutorialAppSchema.Posts
        SET PostTitle = @PostTitle,
            PostContent = @PostContent,
            PostUpdated = GETDATE()
        WHERE PostId = @PostId
            AND UserId = @UserId; -- ownership check, in the database
    END
END
GO
```

The `AND UserId = @UserId` on the update is the IDOR protection from Module 5, enforced at the data layer. Defence in depth:

even if a controller forgets the check, the database updates zero rows.

ON THE JOB `GETDATE()` is server-local time. `GETUTCDATE()` is the correct choice for stored timestamps, for the same reason `DateTime.UtcNow` is correct in C#. Azure SQL servers run in UTC; your local SQL Server runs in Egypt time. Timestamps that shift by two hours between environments are a genuinely confusing bug to chase.

6.8 Calling procedures from Dapper

```
// Simple
var users = connection.Query<User>({
    "TutorialAppSchema.spUsers_Get",
    new { UserId = userId },
    commandType: CommandType.StoredProcedure
});

// With DynamicParameters, including an output parameter
var parameters = new DynamicParameters();
parameters.Add("@FirstName", user.FirstName, DbType.String);
parameters.Add("@UserId", dbType: DbType.Int32, direction: ParameterDirection.Output);

connection.Execute("TutorialAppSchema.spUser Upsert", parameters,
    commandType: CommandType.StoredProcedure);

int newId = parameters.Get<int>("@UserId");
```

TRAP Omitting `commandType: CommandType.StoredProcedure` makes Dapper send the procedure name as a raw SQL statement. The error message is unhelpful. If a procedure call fails with a syntax error, check this first.

Practice — Module 6

1. **First procedure.** Write `spUsers_Get` returning all users. Execute it. Then alter it to add a column, using `CREATE OR ALTER`, and explain why that form matters in a deploy script.
2. **Naming.** Write down why `sp_GetUsers` is a bad name and what SQL Server does differently with it.
3. **Parameters.** Add a required `@UserId`. Call it with a named argument and with a positional one, and say why named is safer.
4. **Output parameter.** Write an insert procedure returning the new `UserId` via an `OUTPUT` parameter. Use the correct identity function and say why the alternative is unsafe.
5. **Nullable parameters.** Convert `spUsers_Get` to take `@UserId` and `@Active`, both defaulting to `NULL`. Verify all four call combinations work.
6. **Parameter sniffing.** Read your nullable-parameter procedure's execution plan for `@UserId = 1` and then for `@UserId = NULL`. Add `OPTION (RECOMPILE)` and compare. Write two sentences on what problem this solves.
7. **Joins.** Extend `spUsers_Get` to `LEFT JOIN` salary and job info. Confirm users missing those rows still appear.
8. **OUTER APPLY.** Write a query returning every user with their single most recent post, including users with no posts. Then write the same thing with `ROW_NUMBER()` and say which you find clearer.
9. **Temp table.** Write a procedure that loads users and salaries into a temp table, then queries it twice. Rewrite it with a CTE and note what changes about how many times the underlying query runs.
10. **Upsert.** Write `spUser_Upsert` handling the main record and both peripheral tables, insert and update.
11. **Make it atomic.** Wrap that upsert in a transaction with `TRY / CATCH` and `THROW`. Force a failure on the second insert and confirm the first is rolled back.
12. **Race condition.** Describe in writing what happens if two connections call your upsert simultaneously for the same new user, and name one way to prevent it.
13. **Delete.** Write `spUser_Delete` removing the peripheral rows and the user. Deliberately reverse the order first and record the foreign key error.
14. **Posts.** Write `spPosts_Get` with three nullable parameters serving all-posts, by-user, by-id and search. Verify each path.
15. **Ownership.** Write `spPost_Upsert` with the `AND UserId = @UserId` ownership check on update. Attempt to update another user's post and confirm zero rows are affected.
16. **UTC.** Change every `GETDATE()` to `GETUTCDATE()` and explain the bug that prevents.
17. **From C#.** Call `spUsers_Get` from Dapper with `CommandType.StoredProcedure`. Then omit that argument and record the error, so you recognise it later.
18. **Output from Dapper.** Call your insert procedure from Dapper and read the new ID back from the output parameter.

Self-check — Module 6

1. Give three arguments for stored procedures and two against.
2. Why is the security argument the strongest one in an enterprise context?
3. Why must you not prefix a procedure with `sp_`?
4. What does `CREATE OR ALTER` solve?
5. Write the nullable-parameter `WHERE` pattern from memory and explain how it reads.
6. What is parameter sniffing, what symptom does it produce, and what is the usual fix?
7. What can `OUTER APPLY` do that a `LEFT JOIN` cannot?
8. What is the difference between `CROSS APPLY` and `OUTER APPLY`?
9. When would you choose a temp table over a CTE?
10. What is an upsert and why does it suit a single-form UI?
11. What two defects does the course's upsert have, and how would you fix each?
12. Why must child rows be deleted before parent rows?
13. Why does the ownership check belong in the procedure as well as the controller?
14. `GETDATE()` or `GETUTCDATE()` — which, and why?
15. What happens if you forget `CommandType.StoredProcedure` in Dapper?
16. How do you read an output parameter back through `DynamicParameters`?

Module 7 — API Advanced

Course section 10. Lectures 111-127. 2h 51m. **This is the section you have not watched at all, and it is the most important one for a delivery project.** Everything before it teaches you to make an API work. This section teaches you to make one that survives code review, refactoring and a second developer.

7.1 What “advanced” means here (L111 API Advanced Theory — 2 min, L112 Stored Procedures Script — 1 min)

Sections 7 and 8 produced a working API with a real defect: **the same knowledge is repeated everywhere**. Column lists appear in every method. Every filter needs its own endpoint. Adding a field to `User` means editing five files.

Section 10 rebuilds the whole thing around three ideas:

1. **Consolidate endpoints.** One `UserComplete` endpoint replaces several, using nullable parameters end to end.
2. **Build queries dynamically.** `DynamicParameters` on the C# side, nullable parameters on the SQL side, so one code path serves many query shapes.
3. **Make SQL reusable.** Stop hand-writing repeated SQL strings; generate them from one definition.

L112 provides the full stored-procedure script for the section. Run it before starting — the C# in the rest of the section depends on procedures existing.

ON THE JOB This is the difference between a graduate who can build a feature and one who can maintain a codebase. In a delivery project the code you write is read and modified by others for years. **The rewrite in this section is the part of the course a senior developer would care about.** When you are asked in a review “why did you structure it this way?”, the reasoning here is your answer.

7.2 UserComplete — consolidating endpoints (L113 UserComplete Setup and Get — 25 min, the longest lecture in the section)

The old design needed `GetUsers`, `GetSingleUser`, `GetActiveUsers` — three endpoints, three SQL strings, three places to change.

The new design is one:

```
[HttpGet("GetUsers/{userId}/{isActive}")]
public IEnumerable<UserComplete> GetUsers(int userId, bool isActive)
{
    string sql = "EXEC TutorialAppSchema.spUsers_Get";
    string stringParameters = "";
    DynamicParameters sqlParameters = new DynamicParameters();

    if (userId != 0)
    {
        stringParameters += ", @UserId = @UserIdParameter";
        sqlParameters.Add("@UserIdParameter", userId, DbType.Int32);
    }
    if (isActive)
    {
        stringParameters += ", @Active = @ActiveParameter";
        sqlParameters.Add("@ActiveParameter", isActive, DbType.Boolean);
    }

    if (stringParameters.Length > 0)
    {
        sql += stringParameters.Substring(1); // strip the leading comma
    }

    return _dapper.LoadDataWithParameters<UserComplete>(sql, sqlParameters);
}
```

Read this carefully, because the two-level naming is the part people miss:

- `@UserId` is the **stored procedure’s** parameter name.
- `@UserIdParameter` is the **Dapper** parameter name.
- The generated SQL is `EXEC spUsers_Get @UserId = @UserIdParameter`, and Dapper substitutes a real `SqlParameter` for `@UserIdParameter`.

The distinct names exist to keep the two layers unambiguous. **The value is still fully parameterised** — only the parameter names, which come from your own code, are concatenated. No user input ever reaches the SQL text.

`UserComplete` is a flattened model joining `Users`, `UserSalary` and `UserJobInfo`:

```

public class UserComplete
{
    public int      UserId      { get; set; }
    public string   FirstName   { get; set; } = string.Empty;
    public string   LastName    { get; set; } = string.Empty;
    public string   Email       { get; set; } = string.Empty;
    public string   Gender      { get; set; } = string.Empty;
    public bool     Active      { get; set; }
    public string   JobTitle    { get; set; } = string.Empty;
    public string   Department  { get; set; } = string.Empty;
    public decimal  Salary      { get; set; }
}

```

The matching `DataContextDapper` method:

```

public IEnumerable<T> LoadDataWithParameters<T>(string sql, DynamicParameters parameters)
{
    IDbConnection dbConnection =
        new SqlConnection(_config.GetConnectionString("DefaultConnection"));
    return dbConnection.Query<T>(sql, parameters);
}

public T LoadDataSingleWithParameters<T>(string sql, DynamicParameters parameters)
{
    IDbConnection dbConnection =
        new SqlConnection(_config.GetConnectionString("DefaultConnection"));
    return dbConnection.QuerySingle<T>(sql, parameters);
}

public bool ExecuteSqlWithParameters(string sql, DynamicParameters parameters)
{
    IDbConnection dbConnection =
        new SqlConnection(_config.GetConnectionString("DefaultConnection"));
    return dbConnection.Execute(sql, parameters) > 0;
}

```

TRAP — the sentinel value problem The course uses `userId != 0` and `isActive` as “was this supplied?” tests. That means **0 and false are unrepresentable as real filter values**. You cannot search for `Active = false` — the code treats it as “no filter”.

The correct approach is nullable parameters with query strings:

```

[HttpGet("GetUsers")]
public IEnumerable<UserComplete> GetUsers(int? userId = null, bool? isActive = null)
{
    if (userId.HasValue) { ... }
    if (isActive.HasValue) { ... }
}

```

`int?` and `bool?` distinguish “not supplied” from “supplied as zero/false”. This is a real bug in the course’s approach and a good thing to spot — if you can explain why in an interview, you have understood the lecture better than someone who just copied it.

Query-string parameters (`/GetUsers?userId=5&isActive=true`) are also more idiomatic REST for optional filters than stuffing them into the route path.

7.3 The rewritten endpoints (L114 *UserComplete Upsert*, L115 *User Delete*, L116 *Posts Get*, L117 *My Posts*, L118 *Post Upsert*, L119 *Post Delete*)

Each follows the same shape. The upsert:

```
[HttpPut("UpsertUser")]
public IActionResult UpsertUser(UserComplete user)
{
    string sql = @"EXEC TutorialAppSchema.spUser_Upsert
        @FirstName = @FirstNameParameter,
        @LastName = @LastNameParameter,
        @Email = @EmailParameter,
        @Gender = @GenderParameter,
        @JobTitle = @JobTitleParameter,
        @Department = @DepartmentParameter,
        @Salary = @SalaryParameter,
        @Active = @ActiveParameter,
        @UserId = @UserIdParameter";

    DynamicParameters sqlParameters = new DynamicParameters();
    sqlParameters.Add("@FirstNameParameter", user.FirstName, DbType.String);
    sqlParameters.Add("@LastNameParameter", user.LastName, DbType.String);
    sqlParameters.Add("@EmailParameter", user.Email, DbType.String);
    sqlParameters.Add("@GenderParameter", user.Gender, DbType.String);
    sqlParameters.Add("@JobTitleParameter", user.JobTitle, DbType.String);
    sqlParameters.Add("@DepartmentParameter", user.Department, DbType.String);
    sqlParameters.Add("@SalaryParameter", user.Salary, DbType.Decimal);
    sqlParameters.Add("@ActiveParameter", user.Active, DbType.Boolean);
    sqlParameters.Add("@UserIdParameter", user.UserId, DbType.Int32);

    if (_dapper.ExecuteSqlWithParameters(sql, sqlParameters))
    {
        return Ok();
    }
    throw new Exception("Failed to Upsert User");
}
```

Verbose — and lecture 127 exists precisely to fix that.

“My Posts” (L117 — 9 min)

```
[HttpGet("MyPosts")]
public IEnumerable<Post> GetMyPosts()
{
    string sql = "EXEC TutorialAppSchema.spPosts_Get @UserId = @UserIdParameter";
    DynamicParameters sqlParameters = new DynamicParameters();
    sqlParameters.Add("@UserIdParameter",
        this.User.FindFirst("userId")?.Value, DbType.Int32);

    return _dapper.LoadDataWithParameters<Post>(sql, sqlParameters);
}
```

The user ID comes from the **token**, not from the route. There is no way for a client to ask for someone else’s posts through this endpoint, because the identity is not an input. This is the IDOR protection from Module 5 expressed as an architectural choice rather than a check.

ON THE JOB Endpoints shaped around “the current user” rather than “a user ID” are structurally safer. `/MyPosts` cannot be attacked by changing a number; `/Posts/{userId}` must be defended. Where the requirement allows it, prefer the former. This is a design point worth raising in a review, and it will land well.

7.4 Auth rewritten (L120 Auth Register — 21 min, L121 Reset Password — 10 min, L122 Login — 11 min)

The auth flow is rebuilt on stored procedures with `DynamicParameters` throughout, and `spRegistration_Upsert` handles both registration and password reset — because both operations do the same thing: generate a new salt, derive a new hash, write both to `Auth`.

```
CREATE PROCEDURE TutorialAppSchema.spRegistration_Upsert
    @Email NVARCHAR(50),
    @PasswordHash VARBINARY(MAX),
    @PasswordSalt VARBINARY(MAX)
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM TutorialAppSchema.Auth WHERE Email = @Email)
    BEGIN
        INSERT INTO TutorialAppSchema.Auth (Email, PasswordHash, PasswordSalt)
        VALUES (@Email, @PasswordHash, @PasswordSalt);
    END
    ELSE
    BEGIN
        UPDATE TutorialAppSchema.Auth
        SET PasswordHash = @PasswordHash, PasswordSalt = @PasswordSalt
        WHERE Email = @Email;
    END
END
GO
```

```
[AllowAnonymous]
[HttpPut("ResetPassword")]
public IActionResult ResetPassword(UserForLoginDto userForSetPassword)
{
    if (_authHelper.SetPassword(userForSetPassword))
    {
        return Ok();
    }
    throw new Exception("Failed to update password");
}
```

TRAP — read this one carefully, it is the most serious issue in the course

A reset-password endpoint marked `[AllowAnonymous]` that takes an email and a new password, with **no verification of any kind**, means anyone who knows a user's email address can set their password and take over the account. As written, this is a complete authentication bypass.

It is written this way to keep the lecture focused on the upsert mechanics, and in a tutorial that is defensible. **Do not carry this pattern into anything real, and if you see it in a codebase, raise it immediately.**

A correct password reset flow:

1. User requests a reset by email address.
2. Server generates a cryptographically random, **single-use, short-lived** token, stores a hash of it against the account, and emails a link containing it.
3. Server returns a generic "if that address exists, we have sent a link" — regardless of whether the account exists, to avoid user enumeration.
4. User follows the link; server validates the token, confirms it is unused and unexpired.
5. Password is updated; the token is invalidated; all existing sessions or refresh tokens are revoked; a confirmation email is sent.

For a *logged-in* password change, require the **current password** as well, so a stolen session cannot be used to lock the real owner out.

Being able to articulate this is worth a great deal in an interview or a design review. Broken authentication sits in the OWASP Top 10 for a reason.

7.5 Cleanup (L123 Cleanup — 13 min)

Deleting the superseded controllers, unused models and dead SQL.

This lecture is more valuable than its title suggests. Dead code is not neutral: it gets read, it gets copied, it gets maintained, it appears in searches, and it misleads whoever joins next. Deleting it is real work.

ON THE JOB Two habits worth forming now.

Delete dead code rather than commenting it out. Git remembers everything. A commented-out block tells the next developer nothing about whether it is a work in progress, a rollback plan or forgotten debris.

Do cleanup in its own commit. Mixing deletions with behaviour changes makes a pull request much harder to review — the reviewer cannot tell what is a move and what is a change. Separate commits, separate concerns. Your reviewers will thank you and your PRs will merge faster.

7.6 DynamicParameters in depth (L124 User Controller DynamicParameters — 20 min)

This lecture consolidates the technique across the whole `UserController`.

```
DynamicParameters sqlParameters = new DynamicParameters();

// Explicit type - recommended
sqlParameters.Add("@UserIdParameter", userId, DbType.Int32);
sqlParameters.Add("@NameParameter", name, DbType.String);
sqlParameters.Add("@SalaryParameter", salary, DbType.Decimal);
sqlParameters.Add("@ActiveParameter", active, DbType.Boolean);

// Output parameter
sqlParameters.Add("@NewIdParameter", dbType: DbType.Int32,
    direction: ParameterDirection.Output);

// From an existing object
DynamicParameters fromObject = new DynamicParameters(user);

// Read a value back after execution
int newId = sqlParameters.Get<int>("@NewIdParameter");
```

TRAP — specify DbType explicitly Without it, Dapper infers the type, and the inference is not always what you want. The classic failure: a C# `string` is inferred as `NVARCHAR(MAX)`, which cannot use an index on an `NVARCHAR(50)` column and forces an implicit conversion — turning an index seek into a full scan. On a large table this is the difference between milliseconds and seconds.

For strings you can be more precise still: `sqlParameters.Add("@EmailParameter", email, DbType.String, size: 50);`

This is a genuinely good performance observation to be able to make in a code review.

The security question, answered clearly

Building SQL strings by concatenation is what Module 3 told you never to do. Why is this acceptable?

Because only names you control are concatenated; every value is parameterised. The strings appended are literals from your own source code (`", @UserId = @UserIdParameter"`), never user input. The user's actual data arrives through `DynamicParameters`, which produces real `SqlParameter` objects sent separately from the query text.

The rule to carry with you: **structure may be built from your own code; values must always be parameters.** If you ever find yourself concatenating something that came from an HTTP request into a SQL string, stop.

7.7 Reusable SQL (L127 Reusable SQL — 15 min, the payoff of the whole section)

The upsert method in 7.3 has nine near-identical lines. Lecture 127 generates them:

```
public class ReusableSql
{
    private readonly DataContextDapper _dapper;

    public ReusableSql(IConfiguration config)
    {
        _dapper = new DataContextDapper(config);
    }

    public bool UpsertUser(UserComplete user)
    {
        string sql = @"EXEC TutorialAppSchema.spUser_Upsert
            @FirstName = @FirstNameParameter,
            @LastName = @LastNameParameter,
            @Email = @EmailParameter,
            @Gender = @GenderParameter,
            @JobTitle = @JobTitleParameter,
            @Department = @DepartmentParameter,
            @Salary = @SalaryParameter,
            @Active = @ActiveParameter,
            @UserId = @UserIdParameter";

        DynamicParameters sqlParameters = new DynamicParameters();
        sqlParameters.Add("@FirstNameParameter", user.FirstName, DbType.String);
        sqlParameters.Add("@LastNameParameter", user.LastName, DbType.String);
        sqlParameters.Add("@EmailParameter", user.Email, DbType.String);
        sqlParameters.Add("@GenderParameter", user.Gender, DbType.String);
        sqlParameters.Add("@JobTitleParameter", user.JobTitle, DbType.String);
        sqlParameters.Add("@DepartmentParameter", user.Department, DbType.String);
        sqlParameters.Add("@SalaryParameter", user.Salary, DbType.Decimal);
        sqlParameters.Add("@ActiveParameter", user.Active, DbType.Boolean);
        sqlParameters.Add("@UserIdParameter", user.UserId, DbType.Int32);

        return _dapper.ExecuteSqlWithParameters(sql, sqlParameters);
    }
}
```

Registered once, injected wherever needed. Every controller that upserts a user calls `_reusableSql.UpsertUser(user)` — one line instead of twenty, and one place to change when a column is added.

The reflection-based version

Dominic goes further, generating the parameter list from the model's properties:

```
public bool UpsertUser(UserComplete user)
{
    string sql = "EXEC TutorialAppSchema.spUser_Upsert";
    string parameters = "";
    DynamicParameters sqlParameters = new DynamicParameters();

    foreach (PropertyInfo property in user.GetType().GetProperties())
    {
        object? value = property.GetValue(user);
        parameters += $", @{property.Name} = @{property.Name}Parameter";
        sqlParameters.Add($"@{property.Name}Parameter", value);
    }

    sql += parameters.Substring(1);
    return _dapper.ExecuteSqlWithParameters(sql, sqlParameters);
}
```

Elegant — and worth understanding both for what it teaches and for where it goes wrong.

Reflection inspects types at runtime. `GetType().GetProperties()` returns every public property; `GetValue()` reads each one.

ON THE JOB — a balanced view of the reflection version

In its favour: adding a property to the model automatically adds it to the query. Zero duplication.

Against: - **Reflection is slow** relative to direct access. Irrelevant for one call; measurable in a hot path. `DbType` inference is also lost, bringing back the `NVARCHAR(MAX)` problem from 7.6. - **It is implicit.** Adding a property to the model silently changes the SQL. If the procedure has no matching parameter, you get a runtime error nobody predicted from reading the code. - **Property order is not guaranteed** by reflection. Positional dependence would break. - **It couples the model to the procedure signature.** They must stay in lockstep, with nothing enforcing it.

My advice for a delivery project: use the *explicit* `ReusableSql` version. It gives you most of the benefit — one place to change, no duplication across controllers — while remaining greppable, debuggable and obvious to the next reader. Reserve reflection for genuinely generic infrastructure, and note that in a codebase using Dapper heavily, `Dapper.Contrib` or source generators solve the same problem with fewer downsides.

If you are asked about this in a review, “I chose the explicit version because the SQL is visible at the call site and the types are declared” is a strong answer.

7.8 The Advanced Assignment (L125 — 1 min, L126 Solution — 5 min)

Apply everything: consolidate remaining endpoints, convert to `DynamicParameters`, extract reusable SQL, delete what is superseded.

Attempt it before watching the solution. It is a better assessment of whether you can work at project level than anything else in the course.

Practice — Module 7

1. **Setup.** Run the L112 stored-procedure script. Confirm each procedure exists and executes.
2. **UserComplete.** Create the `UserComplete` model and a joined `spUsers_Get` returning all its columns. Confirm Dapper maps every property.
3. **Consolidation.** Replace `GetUsers` and `GetSingleUser` with one endpoint using conditional `DynamicParameters`. Verify: no filter, `userId` only, `isActive` only, both.
4. **The sentinel bug.** With the course’s `userId != 0` approach, try to filter for `Active = false`. Confirm it is impossible. Rewrite using `int?` and `bool?` with query-string parameters and confirm it now works. Write two sentences on why nullable types fix it.
5. **Data context.** Add `LoadDataWithParameters<T>`, `LoadDataSingleWithParameters<T>` and `ExecuteSqlWithParameters` to `DataContextDapper`.
6. **Upsert.** Build the `UpsertUser` endpoint with the full `DynamicParameters` list. Confirm it both inserts and updates.
7. **DbType.** Add a user with `DbType.String` specified and again without. Look at both execution plans against an indexed `NVARCHAR(50)` column and identify the difference.
8. **MyPosts.** Build `/MyPosts` taking the user ID from the token. Then attempt to retrieve another user’s posts through it and confirm there is no way to.
9. **Security reasoning.** Write a short paragraph, in your own words, explaining why the string concatenation in this section is safe when the concatenation in Module 4 was not. You will be asked this.
10. **Auth rewrite.** Convert register and login to stored procedures with `DynamicParameters`.
11. **The reset-password flow.** Implement the course’s `[AllowAnonymous]` reset endpoint. Then, as a different “user”, reset someone else’s password and confirm you have taken over the account. Write down the five steps of a correct reset flow.
12. **Fix it.** Implement a token-based reset: a single-use, expiring token stored hashed against the account, validated before the password changes. You do not need real email — log the token.
13. **Cleanup.** Delete every superseded controller, model and SQL string. Do it in a separate commit from any behaviour change, and check nothing broke.
14. **ReusableSql — explicit.** Extract the upsert into a `ReusableSql` class, register it, and call it from two different controllers.
15. **ReusableSql — reflection.** Implement the reflection version. Add a property to `UserComplete` that the procedure does not accept and record the runtime error. Write three sentences on which version you would use on a real project and why.
16. **The assignment.** Complete L125 unaided: every remaining endpoint consolidated, parameterised, and using reusable SQL.
17. **Pagination.** Add `page` and `pageSize` to your consolidated users endpoint using `OFFSET / FETCH NEXT`, returning a total count. The course does not do this; a real project requires it.

Self-check — Module 7

1. What three problems does Section 10 set out to fix?
2. Why are there two parameter names (`@UserId` and `@UserIdParameter`)?
3. Explain precisely why building SQL with `DynamicParameters` is safe. What is concatenated and what is not?
4. What is the sentinel-value bug in `userId != 0`, and how do nullable types fix it?
5. Why are query-string parameters more appropriate than route segments for optional filters?
6. Why is `/MyPosts` structurally safer than `/Posts/{userId}`?
7. What does `DbType.String` without a size cause, and what is the performance consequence?
8. Why should dead code be deleted rather than commented out?
9. Why should cleanup be a separate commit?
10. What is wrong with an `[AllowAnonymous]` reset-password endpoint that takes only an email and a new password?
11. List the steps of a correct password reset flow.
12. When changing a password for a logged-in user, what extra thing should you require, and why?
13. What is reflection, and what do `GetProperties()` and `GetValue()` do?
14. Give three reasons to prefer the explicit `ReusableSql` over the reflection version.
15. What breaks if you add a property to the model in the reflection version?
16. What does `ParameterDirection.Output` let you do, and how do you read the value back?

Module 8 — Azure Deployment and Project Readiness

Course sections 11 and 12. Lectures 128-134. 1h 07m — plus material the course does not cover that your first project will assume you know.

8.1 Azure App Service (L128 — 17 min)

Azure App Service is platform-as-a-service hosting for web applications. Microsoft manages the OS, the runtime, patching, load balancing and scaling; you deploy code.

The hierarchy:

- **Subscription** — the billing boundary.
- **Resource Group** — a logical container for related resources. Everything for one application goes in one group, so it can be deployed, secured and deleted as a unit.
- **App Service Plan** — the compute you pay for: tier, size, region. Multiple apps can share one plan.
- **App Service** — the application itself.

Tiers: **Free / Shared** (development only, no custom domain SSL, cold starts), **Basic** (small production), **Standard** (autoscale, staging slots, daily backups), **Premium** (more power, VNet integration), **Isolated** (dedicated environment, for strict compliance).

ON THE JOB You will almost certainly not create Azure resources by hand on a client project. Infrastructure is defined as code — **Bicep** or **Terraform** — and deployed through a pipeline, so that environments are reproducible and auditable. The portal clicking in this lecture is for learning what the resources *are*.

Two things to ask in week one: “**Is infrastructure defined in Bicep or Terraform, and where does it live?**” and “**Who has permission to deploy to which environment?**” On regulated client work, developers frequently cannot deploy to production at all — a release manager does, from a pipeline, with an approval gate.

Publishing

```
dotnet publish -c Release -o ./publish
```

Deployment options: VS Code’s Azure extension (what the course uses), Visual Studio publish, `az webapp deploy` from the CLI, ZIP deploy, GitHub Actions, or Azure DevOps Pipelines.

ON THE JOB Right-click-publish from an IDE is not how a project deploys. It is unrepeatable, unaudited, and bypasses testing. Every real engagement uses CI/CD: commit triggers build, tests run, artefact is produced once, and that same artefact is promoted through Dev → Test → Staging → Production.

PwC works predominantly in the Microsoft ecosystem, so **Azure DevOps Pipelines** (YAML) or **GitHub Actions** are the likely tools. Learning to read a pipeline YAML file is high-value and takes an afternoon. Being able to say “I looked at the pipeline and I can see where the build and test stages are” in your first week is a genuinely good impression.

8.2 Azure SQL Database (L129 Azure Database Creation — 22 min, L130 Azure SQL Seed — 6 min)

Azure SQL Database is managed SQL Server: no OS, no patching, automated backups, built-in high availability.

Differences from local SQL Server that will catch you:

Local SQL Server	Azure SQL Database
Windows Authentication	Entra ID or SQL Authentication
Unrestricted network access	Firewall rules — your IP must be allowed
<code>USE DatabaseName</code>	Not supported; connect directly to the database
Cross-database queries	Not supported (use Elastic Query)
SQL Agent jobs	Not available (use Elastic Jobs or Azure Functions)
Server-level settings	Mostly unavailable
You manage backups	Automatic, point-in-time restore

Pricing models: **DTU** (a blended compute/memory/IO unit — simple, older) and **vCore** (explicit CPU and memory — more transparent, supports Azure Hybrid Benefit). **Serverless** pauses when idle, which is ideal for a development database and terrible for one that must respond instantly.

Firewall. The first thing that will block you. Add your client IP, and enable “Allow Azure services and resources to access this server” so your App Service can connect. Your home IP changes; expect to re-add it.

TRAP “Allow Azure services” permits *every* Azure resource, including other tenants’, to reach your server at the network level. It is acceptable for learning. For production, use **Private Endpoints** or VNet integration so the database has no public endpoint at all. This is a standard finding in a cloud security review.

Seeding (L130): run the same scripts against Azure SQL through SSMS or the VS Code extension. Note that `GO` works in those tools but still cannot be sent through Dapper.

8.3 Connecting and deploying (L131 Azure Database Connection — 5 min, L132 Azure Deploy with Database Connection — 6 min)

An Azure SQL connection string:

```
Server=tcp:myserver.database.windows.net,1433;  
Initial Catalog=mydatabase;  
Persist Security Info=False;  
User ID=myadmin;  
Password=...;  
MultipleActiveResultSets=False;  
Encrypt=True;  
TrustServerCertificate=False;  
Connection Timeout=30;
```

`Encrypt=True` is mandatory. `TrustServerCertificate=False` is correct — Azure has a real certificate.

Where the connection string goes. Not in `appsettings.json`. In App Service → Configuration → Connection strings. Those values override `appsettings.json` automatically, because App Service surfaces them as environment variables and environment variables sit higher in the configuration precedence order from Module 3. **No code change is required.**

ON THE JOB — the properly modern answer Better than a password in App Service configuration: **Managed Identity**. The App Service gets an Azure AD identity, that identity is granted access to the SQL database, and the connection string contains no credential at all:

```
Server=tcp:myserver.database.windows.net,1433;  
Initial Catalog=mydatabase;  
Authentication=Active Directory Default;  
Encrypt=True;
```

No secret exists to leak, rotate or commit. For anything that must remain a secret — third-party API keys, certificates — use **Azure Key Vault** with the App Service’s managed identity granted read access.

If you are asked in an interview or a design discussion how you would handle database credentials in Azure, “Managed Identity, falling back to Key Vault for secrets that cannot be eliminated” is the answer that marks you as current.

Deployment checklist

1. Set `ASPNETCORE_ENVIRONMENT` correctly (`Production` , not `Development`).
2. Disable Swagger in production, or protect it behind authentication.
3. Configure the production CORS policy with real origins — never `AllowAnyOrigin`.
4. Enforce HTTPS.
5. Confirm no secrets are in `appsettings.json` or source control.
6. Enable Application Insights for logging and telemetry.
7. Set up health check endpoints.
8. Confirm the firewall allows the App Service and nothing more than necessary.

TRAP Leaving `ASPNETCORE_ENVIRONMENT=Development` in a deployed environment exposes the developer exception page — full stack traces, file paths and sometimes configuration values — to anyone who can trigger an error. It is a genuine information-disclosure vulnerability and a routine penetration-test finding.

8.4 Deleting resources (L133 — 9 min)

Delete the **resource group** and everything inside it goes with it. This is why grouping resources by application matters.

ON THE JOB Cloud cost is a real project concern and one graduates are rarely conscious of. An App Service Plan bills whether or not anything is deployed on it. A forgotten Standard-tier plan costs money every month indefinitely. On a client engagement this is billable cost attributed to the project, and someone notices.

Habits: use your own resource group for experiments; tag resources with owner and purpose; delete what you are finished with; set a budget alert on any personal subscription. Being cost-conscious is noticed favourably and costs you nothing.

8.5 What the course does not cover — and your project will expect

This is the gap between “completed the course” and “ready for a delivery project”. None of it is in the 134 lectures.

Testing

The course writes no tests at all. Your project will require them.

```
dotnet new xunit -n DotnetAPI.Tests
dotnet add reference ../DotnetAPI/DotnetAPI.csproj
dotnet add package Moq
dotnet add package FluentAssertions
```

```
public class UserControllerTests
{
    [Fact]
    public void GetSingleUser_WhenUserExists_ReturnsUser()
    {
        // Arrange
        var mockRepo = new Mock<IUserRepository>();
        mockRepo.Setup(r => r.GetSingleUser(1))
            .Returns(new User { UserId = 1, FirstName = "Yassin" });
        var controller = new UserEFController(mockRepo.Object, mapper);

        // Act
        var result = controller.GetSingleUser(1);

        // Assert
        result.FirstName.Should().Be("Yassin");
    }

    [Theory]
    [InlineData(0)]
    [InlineData(-1)]
    public void GetSingleUser_WithInvalidId_ReturnsNotFound(int userId) { }
}
```

xUnit is the .NET standard ([Fact] for a single case, [Theory] with [InlineData] for parameterised cases). **Moq** or **NSubstitute** for mocking. **FluentAssertions** for readable assertions.

JAVA BRIDGE xUnit ≈ JUnit 5. [Fact] ≈ @Test. [Theory] + [InlineData] ≈ @ParameterizedTest + @ValueSource. Moq ≈ Mockito. FluentAssertions ≈ AssertJ.

Note that the repository pattern from Module 5 is what makes any of this possible. You cannot unit test a controller with SQL inside it. That is the practical payoff of the abstraction.

Also learn **WebApplicationFactory<Program>** for integration tests that exercise the whole pipeline in memory — the equivalent of Spring's **@SpringBootTest**.

Logging

The course uses **Console.WriteLine** and unhandled exceptions. Production needs structured logging.

```
public class UserController : ControllerBase
{
    private readonly ILogger<UserController> _logger;

    public UserController(ILogger<UserController> logger) => _logger = logger;

    public IActionResult Get(int id)
    {
        _logger.LogInformation("Fetching user {UserId}", id); // structured, not interpolated
        try { }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Failed to fetch user {UserId}", id);
            return StatusCode(500);
        }
    }
}
```

Note the template placeholders rather than string interpolation — that is what makes logs queryable by field rather than only by substring.

Serilog writing to **Application Insights** is the common enterprise stack. Levels: Trace, Debug, Information, Warning, Error, Critical.

TRAP Never log passwords, tokens, full credit card numbers or personal data. Once logged, it is in your aggregation system, your backups, and possibly a third-party SaaS — with a different, usually weaker, access control model than your database. This is a GDPR issue and a standard audit finding.

Global exception handling

```

app.UseExceptionHandler("/error");

// Or middleware returning RFC 7807 ProblemDetails
app.UseExceptionHandler(appError =>
{
    appError.Run(async context =>
    {
        var feature = context.Features.Get<ExceptionHandlerFeature>();
        logger.LogError(feature?.Error, "Unhandled exception");
        context.Response.StatusCode = 500;
        await context.Response.WriteAsJsonAsync(new ProblemDetails
        {
            Status = 500,
            Title = "An error occurred",
            Instance = context.TraceIdentifier // correlation ID for support
        });
    });
});
});

```

This replaces every `throw new Exception("Failed to...")` in the course with a logged, correlated, sanitised response.

API versioning

```

[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/{controller}")]

```

Once a client integrates with your API you cannot change its shape without breaking them. Version from the start.

Health checks

```

builder.Services.AddHealthChecks()
    .AddSqlServer(connectionString);

app.MapHealthChecks("/health");

```

Azure App Service, load balancers and Kubernetes all use these to decide whether an instance is serving.

Git and pull requests

The course says nothing about source control. Your project runs on it.

- **Branching** — most likely GitHub Flow (`feature/*` off `main`) or GitFlow. Ask which.
- **Commit messages** — imperative mood, explain *why*: “Add ownership check to post update endpoint”, not “fixed stuff”.
- **Pull requests** — small, focused, self-reviewed before you request review. A 2,000-line PR gets a rubber stamp; a 200-line PR gets real scrutiny, which is what you want as a graduate.
- **Receiving review comments** — every comment is information, not criticism. Ask “why” when you do not understand. This is the fastest learning loop available to you and the thing most graduates waste.
- **Giving review comments** — read others’ PRs even when not asked. It is how you learn the codebase.

The React side

Your project is .NET **and** React. Your CV shows React with TypeScript and Node.js, so this is your stronger half. The integration points that matter:

- **CORS** — Module 4. First thing that will break.
- **JSON casing** — C# PascalCase serialises to camelCase. Module 3.
- **Auth token storage** — Module 5. Decide deliberately, do not default to `localStorage`.
- **Typed API client** — generate TypeScript types from your Swagger/OpenAPI document (NSwag or openapi-typescript) rather than hand-writing interfaces. They then cannot drift.
- **Proxy in development** — Vite or CRA proxy configuration avoids CORS locally entirely.
- **Error contract** — agree the error shape with the front end early. `ProblemDetails` is the standard and React can handle it uniformly.

Practice — Module 8

1. **Publish locally.** Run `dotnet publish -c Release -o ./publish` and inspect the output. Identify what is and is not included.
2. **App Service.** Create a resource group, an App Service Plan on a free or basic tier, and an App Service. Deploy your API. Confirm it responds.
3. **Environment.** Set `ASPNETCORE_ENVIRONMENT=Development` in App Service, trigger an error, and observe what is returned. Change it to `Production` and observe the difference. Write two sentences on why the first is a vulnerability.
4. **Azure SQL.** Create an Azure SQL database. Add your client IP to the firewall. Connect from SSMS or VS Code. Run your schema and seed scripts.

5. **Connection string.** Move the connection string into App Service configuration. Confirm the deployed app uses it with no code change, and explain which configuration precedence rule makes that work.
6. **Secrets audit.** Search your entire repository, including Git history, for connection strings, `TokenKey` and `PasswordKey`. Confirm none are committed.
7. **CORS in production.** Configure the production policy with your real front-end origin. Confirm a request from an unlisted origin is blocked.
8. **Cost.** Look up the monthly cost of the App Service Plan tier you chose. Set a budget alert. Then delete the resource group and confirm everything is gone.
9. **Tests.** Create an xUnit project. Write three unit tests for a controller action using a mocked `IUserRepository`: the happy path, the not-found path, and a `[Theory]` covering invalid IDs.
10. **Test the untestable.** Try to write a unit test for one of your Module 4 controllers with SQL inside it. Document why you cannot, and what the repository pattern changed.
11. **Logging.** Replace every `Console.WriteLine` and `throw new Exception` with `ILogger` calls using structured templates. Confirm the placeholders appear as separate fields, not concatenated text.
12. **Global exception handler.** Add middleware that logs unhandled exceptions with a correlation ID and returns a sanitised `ProblemDetails`. Confirm no stack trace reaches the client.
13. **Health check.** Add `/health` including a SQL Server check. Stop the database and confirm the endpoint reports unhealthy.
14. **Swagger to TypeScript.** Generate TypeScript interfaces from your API's OpenAPI document. Change a C# property name, regenerate, and watch the front end fail to compile — which is the point.
15. **End to end.** Build a small React page that registers, logs in, stores the token, and calls a protected endpoint with the `Authorization` header. Get it working against your deployed API.
16. **Read a pipeline.** Find any public Azure DevOps or GitHub Actions YAML file for a .NET project. Identify the build, test and deploy stages and write three sentences describing what happens on a commit.

Self-check — Module 8

1. What is the difference between a Resource Group, an App Service Plan and an App Service?
2. Name three differences between local SQL Server and Azure SQL Database.
3. What does “Allow Azure services” actually permit, and why is it not production-safe?
4. Why must `Encrypt=True` be set for Azure SQL?
5. Where should a production connection string live, and why does no code change result?
6. What is Managed Identity and what problem does it eliminate entirely?
7. What is exposed if `ASPNETCORE_ENVIRONMENT=Development` in a deployed environment?
8. Why is right-click-publish unacceptable on a real project?
9. Why does deleting a resource group matter for cost?
10. What makes a controller unit-testable, and which module introduced it?
11. What is the difference between `[Fact]` and `[Theory]`?
12. Why does `LogInformation("User {UserId}", id)` beat `LogInformation($"User {id}")`?
13. Name three things that must never be logged.
14. What does a global exception handler give you that per-action `throw` does not?
15. Why version an API from the beginning?
16. What are health checks used for, by what?
17. Name three integration points between a .NET API and a React front end that commonly break.
18. Why generate TypeScript types from OpenAPI rather than writing them by hand?

Appendix A — Java to C# Quick Reference

Concept	Java	C#
Runtime	JVM	CLR
Package manager	Maven / Gradle	NuGet
Build file	<code>pom.xml</code>	<code>.csproj</code>
Web framework	Spring Boot	ASP.NET Core
ORM	Hibernate / JPA	Entity Framework Core
Lightweight data access	JdbcTemplate	Dapper
DI container	Spring IoC	Built-in <code>IServiceCollection</code>
Testing	JUnit 5	xUnit
Mocking	Mockito	Moq / NSubstitute
Assertions	AssertJ	FluentAssertions
Method naming	<code>camelCase</code>	<code>PascalCase</code>
Private field	<code>firstName</code>	<code>_firstName</code>
Interface naming	<code>UserRepository</code>	<code>IUserRepository</code>
Getter/setter	<code>getX()</code> / <code>setX()</code>	<code>{ get; set; }</code> property
Inheritance	<code>extends</code> / <code>implements</code>	<code>:</code> for both
Parent reference	<code>super</code>	<code>base</code>
Overridable by default	Yes	No — needs <code>virtual</code>
Final class	<code>final</code>	<code>sealed</code>
Package-private	package scope	<code>internal</code> (assembly scope)
Streams	<code>stream().filter().map()</code>	LINQ <code>.Where().Select()</code>
Optional	<code>Optional<T></code>	<code>T?</code> + nullable analysis
try-with-resources	<code>try (var x = ...)</code>	<code>using</code>
Async	<code>CompletableFuture</code>	<code>async</code> / <code>await</code> / <code>Task<T></code>
Annotation	<code>@Component</code>	<code>Attribute</code> [<code>ApiController</code>]
String format	<code>String.format</code>	<code>\$"interpolation"</code>
Collection size	<code>.size()</code> / <code>.length</code>	<code>.Count</code> / <code>.Length</code>
Get element	<code>.get(0)</code>	<code>[0]</code>
Records	<code>record</code>	<code>record</code>
Switch expression	<code>switch -></code>	<code>switch { => }</code>

Appendix B — Command reference

.NET CLI

```
dotnet --version / --list-sdks / --info
dotnet new console|webapi|classlib|xunit|sln|gitignore -n Name
dotnet sln add Project/Project.csproj
dotnet restore | build | run | watch run | clean | test
dotnet add package PackageName [--version X.Y.Z]
dotnet list package [--outdated]
dotnet publish -c Release -o ./publish
dotnet user-secrets init | set "Key" "Value" | list
dotnet dev-certs https --trust
```

Entity Framework

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add NameOfMigration
dotnet ef database update
dotnet ef migrations remove
dotnet ef migrations script
dotnet ef dbcontext scaffold "ConnStr" Microsoft.EntityFrameworkCore.SqlServer
```

Git

```
git switch -c feature/my-change
git add -p # stage selectively – review as you go
git commit -m "Add ownership check to post update"
git push -u origin feature/my-change
git rebase main # keep history linear, if your team does this
git log --oneline --graph
```

Appendix C — Glossary

ADO.NET — the low-level .NET data access layer beneath Dapper and EF. **Assembly** — a compiled `.dll` or `.exe`. The unit of `internal` visibility. **BCL** — Base Class Library, the .NET standard library. **CLR** — Common Language Runtime, the .NET virtual machine. **Claim** — a name/value assertion inside a JWT, e.g. `userId=5`. **CORS** — Cross-Origin Resource Sharing; how a server permits cross-origin browser requests. **CTE** — Common Table Expression, a named temporary result set in a single SQL statement. **Dapper** — micro-ORM mapping SQL results to objects. **DbContext** — EF Core's unit of work and change tracker. **DbSet** — EF Core's queryable representation of a table. **Deferred execution** — LINQ queries do not run until enumerated. **DI** — Dependency Injection. **DTO** — Data Transfer Object; a shape designed for an API operation. **DynamicParameters** — Dapper's imperatively-built parameter collection. **Entity Framework Core** — Microsoft's full ORM. **Idempotent** — repeating the operation produces the same end state. **IDOR** — Insecure Direct Object Reference; trusting a client-supplied ID for authorization. **IL** — Intermediate Language; what C# compiles to. **JIT** — Just-In-Time compilation of IL to native code. **JWT** — JSON Web Token; a signed, self-contained credential. **LINQ** — Language Integrated Query. **Middleware** — an ordered component in the ASP.NET Core request pipeline. **Migration** — a versioned, code-defined schema change in EF Core. **N+1** — one query for a list plus one per item; the classic ORM performance bug. **NuGet** — the .NET package manager. **Over-posting** — a client setting model properties it should not, by binding to an entity. **Parameter sniffing** — SQL Server caching a plan based on the first call's parameter values. **PBKDF2** — a deliberately slow password hashing function. **Pepper** — an application-wide secret mixed into password hashing, stored outside the database. **ProblemDetails** — RFC 7807 standard error response shape. **Reflection** — inspecting types and members at runtime. **Repository pattern** — an interface between business logic and data access. **Salt** — per-user random bytes mixed into a password hash. **Sargable** — a predicate that can use an index. **SCOPE_IDENTITY()** — the last identity value generated in the current scope. **Soft delete** — flagging a row inactive rather than removing it. **Stored procedure** — named, parameterised SQL stored in the database. **T-SQL** — Microsoft's SQL dialect. **Upsert** — update if present, insert if not.

Appendix D — Your first weeks on the project

The course teaches the technology. This is the part that determines how the project actually goes.

Questions to ask in week one

Write these down and get answers. Asking them marks you as someone who thinks about delivery, not just code.

Codebase 1. What .NET version, and is it LTS? 2. Dapper, EF Core, or both? Which for what? 3. Stored procedures or ORM-generated SQL? Are procedures in source control? 4. How do schema changes get deployed — migrations, or reviewed SQL through a DBA? 5. Is there a `NuGet.config` pointing at an internal feed, and how do I authenticate to it? 6. How do I get a working local database from a clean clone?

Process 7. What is the branching strategy? 8. What does a good pull request look like here — size, description, who reviews? 9. What is the definition of done? Tests, documentation, review, deployed to which environment? 10. Where is the CI/CD pipeline and what does it run?

Environment and access 11. Which environments exist, and who can deploy to each? 12. Where do secrets live — Key Vault, App Service config, something else? 13. What is the logging and monitoring stack?

Team 14. Who is my day-to-day point of contact for technical questions? 15. What is the ceremony rhythm — standup, planning, retro? 16. What does the client actually care about most — deadline, quality, scope?

How to spend your first two weeks

Days 1-3: read, do not write. Clone the repository, get it running locally, read the codebase top to bottom. Trace one request end to end — controller, service, repository, database — and draw it. Read the last twenty merged pull requests to learn the team's conventions and what reviewers care about.

Days 4-7: take the smallest ticket available. A bug fix, a copy change, a small validation rule. The goal is not the feature; it is to traverse the whole process once — branch, code, test, PR, review, merge, deploy — while the stakes are low. Discovering the pipeline is broken on a trivial change is a good day. Discovering it on a critical one is not.

Week 2: contribute properly, and ask more questions than you think you should. Graduates consistently under-ask. Every hour you spend stuck on something a colleague could answer in two minutes is an hour the project lost. The rule that works: **try for thirty minutes, then ask** — and when you ask, say what you have already tried.

Delivering successfully, in six points

1. **Communicate early when something slips.** A problem raised on day two is a schedule adjustment. The same problem raised on day nine is a crisis. Nobody is upset about a well-flagged delay; everybody is upset about a surprise.
2. **Small, frequent commits and small pull requests.** A 200-line PR gets a real review, which is how you learn. A 2,000-line PR gets a rubber stamp, which is how bugs ship.
3. **Follow the existing conventions, even ones you disagree with.** Consistency in a codebase is worth more than any individual improvement. If you think something is genuinely wrong, raise it as a question in a retrospective, not unilaterally in a PR.
4. **Write the test.** On a client deliverable, “it works on my machine” is not evidence. A test is.
5. **Treat every review comment as free training.** The reviewer is spending their time making you better. Ask “why” when you do not understand the reasoning — that question is welcomed and it is how you accelerate.
6. **Understand what the client is actually trying to achieve.** The requirement as written is someone's guess at a solution. A developer who understands the underlying business problem catches the cases the specification missed — and that, more than code quality, is what gets a graduate noticed at a firm like this one.

Final assessment

You are ready when you can do all of these without reference material:

- ☐ Explain the difference between .NET Framework, .NET Core and .NET 9, and say which is LTS.
- ☐ Scaffold a web API, add a controller and a DTO, and run it.
- ☐ Explain the three DI lifetimes and pick the right one for a repository.
- ☐ Explain why `UseAuthentication` precedes `UseAuthorization`.
- ☐ Write parameterised Dapper queries and explain precisely why they prevent injection.
- ☐ Write an EF Core `DbContext`, do CRUD, and explain change tracking.
- ☐ Describe the N+1 problem and fix it.
- ☐ Argue both sides of Dapper versus EF Core, and of stored procedures versus ORM.
- ☐ Write a stored procedure with nullable parameters, and explain parameter sniffing.

- ☐ Write the upsert pattern and identify its two defects.
- ☐ Explain hashing versus encryption, and what a salt and a pepper each defeat.
- ☐ Explain the three parts of a JWT and why the payload must contain nothing sensitive.
- ☐ Explain IDOR and the one rule that prevents it.
- ☐ Explain over-posting and how DTOs stop it.
- ☐ Return the correct HTTP status code in every case, including 401 versus 403.
- ☐ Explain CORS and why it breaks a React front end specifically.
- ☐ Say where secrets belong locally, in a deployed environment, and in production.
- ☐ Write a unit test for a controller with a mocked repository.
- ☐ Describe a correct password reset flow.
- ☐ Draw the full request path from React through to a stored procedure and back.

You have my number. Send me your practice work module by module and I will mark it, and bring me the project details the moment you have them so we can narrow this to what you will actually be building.